

Predicting Loan Defaults with Machine Learning: A Study of Class Imbalance, Feature Engineering, and Model Interpretability

By

Derriek Xavier

Thesis submitted to the University of Plymouth
in partial fulfilment of the requirements for the degree of
MSc Data Science and Business Analytics

University of Plymouth
Faculty of Science & Engineering

September 2025

Masters Dissertation licence

This material has been deposited in the Plymouth University Learning & Teaching repository under the terms of the student contract between the students and the Faculty of Science & Engineering

The material may be used for internal use only to support learning and teaching.

Materials will not be published outside of the University and any breaches of this licence will be dealt with following the appropriate University policies.

Copyright Notice

This copy of the thesis has been supplied on condition that anyone who consults it is understood to recognise that its copyright rests with the author and that no quotation from the thesis and no information derived from it may be published without the author's prior written consent.

Abstract

This dissertation develops a pre-loan credit-risk pipeline for loan default prediction on Lending Club (2007–2018). To ensure deployability at origination and avoid data leakage, models use only features available before a lending decision. The study follows a phased design: (i) baseline Logistic Regression and Random Forest; (ii) class-imbalance mitigation with SMOTE/SMOTE-ENN and decision-threshold tuning; (iii) comparison of boosted ensembles (XGBoost, LightGBM) and PCA(Principal Component Analysis) checks; and (iv) an enhanced preprocessing workflow that engineers temporal indicators (issue_year, issue_month), credit-history aggregates (open_acc, pub_rec, total_acc), and delinquency recency (sentinel + missingness flag), alongside cost-sensitive learning. Results show the expected precision–recall trade-off under severe imbalance: resampling and threshold tuning markedly increase recall at the expense of precision. With the enhanced feature set and cost sensitivity, the final LightGBM model achieves $F1 = 0.44$ and $AUC = 0.73$, a realistic and leakage-free benchmark for origination-time screening. SHAP analysis supports interpretability, consistently identifying interest rate, issue year, loan term, and average FICO as dominant drivers.

Word Count: 9004

Acknowledgements

I would like to thank my supervisor, Jason Hughes, for his patient guidance, insightful comments, and continual encouragement at every stage of this dissertation. His insights and advice helped me stay on track during challenging moments.

I am also indebted to my lecturers and tutors from the School of Engineering, Computing and Mathematics at the University of Plymouth, whose teaching has given me the knowledge and ability to carry out this research.

This research uses publicly available data: the Lending Club dataset (2007–2018), obtained via Kaggle. I acknowledge that I did not gather these data myself; they were accessed in accordance with the platform's terms of use and used solely for academic purposes.

List of Contents

1. Introduction	1
1.1 Aims and Objectives	2
2. Literature Review	3
2.1 Traditional Approaches to Credit Scoring.....	4
2.2 Machine Learning Approaches in Loan Default Prediction	6
2.2.1 Decision Trees.....	6
2.2.2 Random Forest.....	7
2.2.3 Gradient Boosting and XGBoost.....	7
2.2.4 LightGBM.....	8
2.2.5 Comparative Findings.....	8
2.3 The Challenge of Class Imbalance in Credit Risk Data	9
2.4 Evaluation Metrics for Loan Default Prediction	10
2.5 Model Interpretability in Credit Risk	12
2.6 Summary of Literature and Research Gap.....	13
3. Methodology.....	15
3.1 Research Design	15
3.2 Dataset.....	17
3.3 Data Preprocessing	18
3.4 Class Imbalance Handling	21
3.5 Model Selection and Training.....	24

3.6 Hyperparameter Tuning	25
3.7 Performance Evaluation	27
3.8 Threshold Analysis	28
3.9 Model Interpretability	30
4. Results	31
4.1 Baseline Model Performance	32
4.2 Resampling with SMOTE and SMOTE-ENN	33
4.3 Gradient Boosting Models	33
4.4 Enhanced Feature Engineering and Cost-Sensitive Learning	34
4.5 Model Interpretability	36
4.6 Summary of Results	38
5. Discussion	38
5.1 Model Performance and Class Imbalance	38
5.2 Boosting Methods and Comparative Findings	39
5.3 Feature Engineering and Temporal Effects	40
5.4 Interpretability and Practical Implications	40
5.5 Realistic Benchmarks and Limitations	41
6. Conclusion	41
7. References	45
Appendix	52
Appendix A – R Code for Exploratory Data Analysis	52

Appendix B – Jupyter notebook code for initial model training.....	59
Appendix C – R Code for enhanced EDA	74
Appendix D – Jupyter notebook for final model evaluation	75

List of Figures

Figure 1 Interest Rate % vs Default.....	19
Figure 2 Correlation heatmap of features.....	20
Figure 3 Imbalance between non defaults and defaults	22
Figure 4 Precision vs Recall with different thresholds	29
Figure 5 Initial results of ML models.....	33
Figure 6 ROC curves on the test set for top models; LightGBM achieves the highest AUC.....	34
Figure 7 Default Rate vs Loan Issue Year	35
Figure 8 Final ML model scores	36
Figure 9. SHAP values for interpretability ranked from most to least impactful	37
Figure 10 SHAP most important features.....	42

1. Introduction

A loan default occurs when a borrower fails to repay a loan according to the pre-agreed terms. Loan default prediction is, therefore a central challenge in credit risk management, as financial institutions must accurately assess the likelihood of default before granting loans. The ability to identify high-risk borrowers in advance enables lenders to minimise losses, manage capital more effectively, and comply with regulatory requirements (Alvi et al., 2024). Conversely, misclassification of risky applicants can lead to significant financial damage and instability in lending portfolios, making the development of robust predictive models a critical priority for banks, fintech companies, and regulators alike (Thomas et al., 2017).

Effective default prediction is therefore necessary for risk management. By identifying potentially high-risk borrowers in advance, lenders can make more informed credit decisions, adjust interest rates or deny risky borrowers access to loans. This task, however, is challenging due to the unpredictable nature of borrower behaviour and a huge range of factors influencing repayment outcomes.

Credit risk assessment has relied on statistical scorecards and linear models, most notably logistic regression, which have been favoured for their transparency and ease of implementation. However, these traditional approaches are limited in their ability to capture the non-linear and high-dimensional relationships often present in borrower data (Malekipirbazari and Aksakalli, 2015). The rapid growth of peer-to-peer lending platforms and the availability of large-scale loan datasets have accelerated the adoption of machine learning methods that can address these shortcomings. Proactive approaches to early default prediction are also gaining

attention, as they allow institutions to intervene before loans fall into arrears (Monje et al., 2025).

Despite these advantages, several considerations remain hugely critical when deploying ML in credit risk assessments, one being the importance of the results being interpretable. Models must provide transparent and fair justification for their predictions. This study attempts to address these challenges by building a comprehensive, transparent and replicable ML pipeline for loan default prediction. Industry reports highlight the growing use of machine learning in fintech credit scoring and the associated governance and explainability challenges (BIS, 2019).

1.1 Aims and Objectives

The aim of this dissertation is to explore the effectiveness of various ML models in predicting loan defaults using historical loan data, with a particular focus on improving the identification and prediction of defaulters through classification techniques. The study evaluates performance based not only on traditional metrics such as accuracy, but also on recall and F1-score, which are more suitable for imbalanced classification problems. The interpretability of the models will also be considered through feature importance analysis.

The key objectives of this research are:

- To build predictive models to identify borrowers who are more likely to default, using classification techniques
- To identify key features that influence loan defaults using exploratory data analysis techniques.

- To compare performance across multiple machine learning algorithms, including Logistic Regression, Random Forest, XGBoost and LightGBM.
- To evaluate model performance using appropriate metrics, including accuracy, recall, precision, F1-score, with an emphasis on recall and F1 score due to the class imbalance of the data set.
- To address class imbalance using oversampling techniques such as SMOTE.
- To interpret and explain model predictions using feature importance analysis and SHAP(Shapley Additive Explanations).

2. Literature Review

Loan default prediction is a central challenge in credit risk management, as financial institutions must accurately assess the likelihood of default before granting loans.

The ability to identify high-risk borrowers in advance enables lenders to minimise losses, manage capital more effectively, and comply with regulatory requirements.

Conversely, misclassification of risky applicants can lead to significant financial damage and instability in lending portfolios. This makes the development of robust predictive models a critical priority for banks, fintech companies, and regulators alike (Thomas et al., 2017).

Historically, credit risk assessment has relied on statistical scorecards and linear models, most notably logistic regression, which have been favoured for their transparency and ease of implementation. However, these traditional approaches are limited in their ability to capture the non-linear and high-dimensional relationships often present in borrower data (Malekipirbazari and Aksakalli, 2015). The rapid growth of peer-to-peer lending platforms and the availability of large-scale loan

datasets have accelerated the adoption of machine learning methods that can address these shortcomings.

Recent studies have demonstrated the effectiveness of advanced ensemble-based algorithms, such as Random Forest, XGBoost, and LightGBM, in modelling complex borrower–lender dynamics and delivering stronger predictive performance compared to logistic regression (Lessmann et al., 2015). These models have proven particularly successful in identifying subtle risk patterns, although their complexity raises concerns about interpretability and compliance in regulated industries.

Another persistent challenge is the class imbalance problem, where non-default loans vastly outnumber defaults, often leading to biased models that favour the majority class. Resampling methods such as SMOTE and its hybrid extensions have been proposed to mitigate this issue and improve the detection of minority cases (Lessmann et al., 2015).

Alongside accuracy and recall, the interpretability of machine learning models has gained increasing importance. Tools such as SHAP provide post-hoc explanations that allow lenders to understand how predictions are made, thereby enhancing trust and accountability in credit scoring systems (Lundberg and Lee, 2017). Against this backdrop, this literature review critically examines the evolution of loan default prediction, beginning with traditional approaches, moving through modern machine learning methods, and exploring how class imbalance, evaluation metrics, and interpretability have been addressed in prior research.

2.1 Traditional Approaches to Credit Scoring

Early developments in credit risk modelling were dominated by statistical approaches, which provided a structured way for lenders to formalise lending

decisions. Among these methods, logistic regression quickly became the most widely adopted model for predicting loan defaults. Its appeal lay in its computational efficiency and ability to handle binary outcomes such as default versus non-default, while producing coefficients that could be interpreted as odds ratios. This interpretability made it highly attractive to financial institutions and regulators, who often required clear justification for why applicants were accepted or rejected (Thomas, 2009). Even in contemporary practice, logistic regression continues to serve as a baseline model in many banks, due to its simplicity which supports regulatory compliance and facilitates model validation.

Other statistical models were also applied in early credit scoring research, most notably linear discriminant analysis (LDA) and probit regression. These techniques are conceptually similar to logistic regression but rely on stronger statistical assumptions, such as normally distributed predictors and equal covariance across classes. While these assumptions may hold in carefully controlled datasets, they often break down in the heterogeneous and unbalanced datasets common in consumer lending (Thomas et al., 2017). As a result, their predictive power was generally weaker and their use less widespread than logistic regression.

Despite their limitations, traditional statistical models remain important benchmarks in credit risk modelling. Their transparency offers a level of accountability that is difficult to replicate in complex machine learning approaches, and their continued use reflects a reliable need for interpretability in high-stakes financial decisions. Indeed, logistic regression remains a standard point of comparison in academic research and a trusted operational tool in industry (Óskarsdóttir et al., 2019).

However, as digital lending platforms and peer-to-peer systems have expanded, the weaknesses of these traditional approaches have become increasingly apparent. Large-scale lending data often exhibits non-linear relationships between borrower attributes and repayment outcomes, patterns that linear models fail to capture. The rise of alternative data sources, such as transaction records and digital footprints, has also introduced high-dimensional feature spaces where traditional techniques are less effective (Malekipirbazari and Aksakalli, 2015). These developments have powered the transition to machine learning methods, which are better equipped to deal with model complexity and heterogeneity in credit risk data.

2.2 Machine Learning Approaches in Loan Default Prediction

Building on the traditional benchmark of logistic regression discussed, this section reviews non-linear machine-learning methods that can model interactions and complex effects common in credit data. The limitations of traditional statistical approaches in modelling complex borrower-lender dynamics have encouraged the adoption of machine learning (ML) techniques in credit risk assessment. Unlike logistic regression and related models, ML methods can capture non-linear relationships, handle high-dimensional data, and improve predictive accuracy. This section reviews the most relevant ML approaches to loan default prediction: decision trees, Random Forests, and boosting algorithms such as XGBoost and LightGBM.

2.2.1 Decision Trees

Decision Trees represent one of the earliest ML approaches applied to credit scoring. They classify borrowers by recursively splitting the dataset into subsets based on feature thresholds, producing a tree structure that is relatively easy to interpret. Their ability to capture non-linear effects provides an advantage over logistic regression,

particularly when borrower behaviour depends on threshold effects, such as income cut-offs or credit utilisation ratios. However, standalone trees are prone to overfitting, where the model learns noise in the training data, and they can be unstable because small changes in data often lead to different splits and tree structures (Mirzamomen, Z and Kangavari, MR, 2017). This instability limits their usability in large-scale loan datasets, although they remain useful for establishing simple, interpretable rule sets. This study did not explore decision trees since there are improved versions such as Random Forests, Boosting algorithms, etc.

2.2.2 Random Forest

Random Forests improve upon decision trees by constructing an ensemble of trees trained on bootstrapped samples of the data and subsets of features. The final prediction is based on the majority vote across trees, reducing the risk of overfitting and improving robustness. This method has been widely adopted in credit risk research, where it has consistently shown stronger predictive performance than logistic regression, particularly in heterogeneous datasets such as peer-to-peer lending platforms (Malekipirbazari and Aksakalli, 2015). Random Forests also provide measures of feature importance, allowing researchers to identify key drivers of default risk, which makes them more interpretable than many other ML methods. However, they are less transparent than a single tree or logistic regression, and the ensemble nature can still limit their acceptance in regulatory environments.

2.2.3 Gradient Boosting and XGBoost

Boosting algorithms take a different ensemble approach, training trees sequentially, where each new tree corrects the errors of its predecessors. XGBoost has emerged as one of the most effective implementations of gradient boosting, optimised for

speed and scalability. In loan default prediction, XGBoost has demonstrated superior predictive performance compared to both Random Forests and logistic regression, often achieving higher recall and precision across imbalanced datasets (Yang et al., 2025). Its popularity stems from its ability to model subtle feature interactions, making it particularly suitable for capturing borrower–lender patterns. However, its complexity introduces challenges in interpretability, as the contribution of individual features to predictions is less transparent without post-hoc tools such as SHAP.

2.2.4 LightGBM

LightGBM, developed more recently by Microsoft, extends the gradient boosting framework by using histogram-based algorithms and leaf-wise tree growth to significantly improve efficiency. It can manage very large datasets with high-dimensional feature spaces more quickly than XGBoost, while achieving comparable predictive performance. Research applying LightGBM to credit risk modelling has shown that it performs particularly well in large-scale loan datasets, providing both speed and accuracy in comparison to earlier boosting methods (Ke et al., 2017). LightGBM's efficiency makes it attractive in operational environments where rapid retraining is necessary, such as online lending platforms. Nevertheless, like XGBoost, its interpretability remains limited without additional explanatory frameworks.

2.2.5 Comparative Findings

Comparative research consistently highlights that ensemble methods outperform traditional statistical models in predictive tasks. Ensemble classifiers, including Random Forests and boosting approaches, have been shown to provide higher classification accuracy and robustness compared to single models (Abellán and

Castellano, 2017). Similarly, boosting models in particular have been found to deliver the strongest performance on imbalanced credit scoring data (Brown and Mues, 2012). More recent benchmarking studies confirm that gradient boosting algorithms, especially XGBoost and LightGBM, remain among the most competitive approaches for loan default prediction, although the trade-off between accuracy and interpretability remains a persistent challenge (Malekipirbazari and Aksakalli, 2015).

2.3 The Challenge of Class Imbalance in Credit Risk Data

A major challenge in loan default prediction is the class imbalance problem, which arises because the number of non-defaulted loans vastly outnumbers defaulted ones. In large lending datasets such as those from peer-to-peer platforms, defaults often make up less than 20% of the total, and in some cases as low as 5%. This imbalance skews model training, leading classifiers to favour the majority class and produce high overall accuracy while failing to correctly identify defaulting borrowers (Sun et al., 2009). In credit risk contexts, such false negatives are costly, as undetected defaults directly translate into financial losses for lenders.

Traditional methods for addressing imbalance include undersampling the majority class or oversampling the minority class. Undersampling reduces the dataset size by removing non-default observations, which can cause a loss of valuable information. The most influential oversampling method is the Synthetic Minority Over-sampling Technique (SMOTE), which generates new synthetic samples by interpolating between existing minority observations. This approach reduces overfitting compared to simple duplication and has been widely applied in credit risk research (Chawla et al., 2002).

While SMOTE improves recall for defaulters, it may also introduce noise by creating synthetic points close to class boundaries. To address this, hybrid techniques combine oversampling with noise reduction. One such method is SMOTE with Edited Nearest Neighbours (SMOTE-ENN), which removes noisy or borderline synthetic points after oversampling. Studies applying SMOTE-ENN in financial datasets show that it often improves both recall and F1-scores compared to standard SMOTE (Batista et al., 2004).

Alternative strategies include cost-sensitive learning, where misclassification costs are incorporated directly into the training process. In the context of credit scoring, this means assigning higher penalties to false negatives than false positives, reflecting the higher financial risk of missed defaults. Cost-sensitive boosting approaches have been shown to outperform sampling methods by embedding imbalance handling within the model itself, while still maintaining predictive accuracy (He and Garcia, 2009).

Overall, addressing class imbalance is critical for fair and effective loan default prediction. Resampling techniques such as SMOTE and SMOTE-ENN remain popular for improving detection of minority cases, but cost-sensitive algorithms provide an increasingly attractive alternative. The choice of strategy depends on the data characteristics and the balance between maximising recall for defaulters and maintaining generalisation performance.

2.4 Evaluation Metrics for Loan Default Prediction

Evaluating the performance of loan default prediction models requires careful selection of metrics, particularly because of the class imbalance problem. Relying solely on overall accuracy can be misleading, as models may achieve high accuracy

by predicting most loans as non-defaults while failing to correctly identify defaulting borrowers (Bekhet and Eletter, 2014).

To address this, researchers commonly use precision and recall, which provide more informative insights into model behaviour on minority cases. Precision measures the proportion of loans predicted as defaults that actually defaulted, reflecting the cost of false positives. Recall, in contrast, measures the proportion of actual defaulters that were correctly identified, capturing the cost of false negatives. In financial contexts, recall is often prioritised, as missing a defaulter typically results in greater losses than incorrectly rejecting a creditworthy applicant (Pandey et al., 2021).

$$Precision = \frac{TP}{TP+FP} \quad Recall = \frac{TP}{TP+FN} \quad Accuracy = \frac{TP+TN}{TP+FP+TN+FN}$$

Where TP = True Positives, FP = False Positives, TN = True Negatives, FN = False Negatives

The F1-score, defined as the harmonic mean of precision and recall, offers a single summary metric that balances both perspectives. It is widely used in credit risk research to provide a more nuanced evaluation of classifiers in the presence of imbalanced classes (Ali et al., 2015).

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Another widely adopted measure is the Area Under the Receiver Operating Characteristic Curve (AUC-ROC), which evaluates the ability of a model to discriminate between defaulters and non-defaulters across all possible thresholds. A higher AUC value indicates stronger ranking capability, making it particularly useful when comparing models independently of a chosen threshold. However, in highly imbalanced scenarios, Precision–Recall (PR) curves may provide a more informative

view, as they focus directly on the performance of the minority default class (Saito and Rehmsmeier, 2015).

A comprehensive evaluation of loan default prediction models requires multiple complementary metrics. While accuracy remains a baseline, precision, recall, F1-score, and AUC-ROC are more reliable indicators of model performance in imbalanced settings, with PR curves offering additional insights for highly skewed datasets.

2.5 Model Interpretability in Credit Risk

While predictive accuracy is a central goal of loan default modelling, the interpretability of models is equally important in financial contexts. Credit decisions directly affect individuals and institutions, and regulators require that such decisions can be explained and justified. Traditional statistical methods such as logistic regression have long been valued for their transparency, as their coefficients directly indicate the influence of borrower characteristics on default risk. However, as financial institutions increasingly adopt complex machine learning methods such as ensemble models and boosting algorithms, the challenge of ensuring interpretability has become more pronounced (Rudin, 2019).

Model interpretability is particularly critical in regulated industries because opaque models pose risks of unfair treatment and regulatory non-compliance. For example, the European Union's General Data Protection Regulation (GDPR) stipulates a "right to explanation" for automated decision-making, reinforcing the importance of using models that can be understood by stakeholders (Goodman and Flaxman, 2017). In practice, this means that even if a complex model such as XGBoost provides higher

predictive accuracy, its adoption may be restricted if its outputs cannot be explained in a clear and auditable way.

To address this issue, researchers have increasingly turned to post-hoc explanation methods that make complex models more interpretable. One widely used approach is SHAP (Shapley Additive Explanations), which assigns each feature a contribution value for individual predictions, based on principles from cooperative game theory. SHAP has been shown to provide consistent and locally accurate explanations of model behaviour in credit risk prediction (Lundberg and Lee, 2017). Another influential technique is LIME (Local Interpretable Model-Agnostic Explanations), which approximates complex models with simpler surrogate models around a given prediction, providing insight into the local decision boundaries (Ribeiro et al., 2016).

Despite the usefulness of these tools, concerns remain about whether post-hoc explanations are sufficient for high-stakes financial applications. Some scholars argue for the development of inherently interpretable models, such as constrained decision trees or scoring systems, that combine predictive performance with built-in transparency. Post-hoc methods may sometimes be misleading, and designing models that are interpretable by design may offer a stronger alternative (Rudin, 2019).

2.6 Summary of Literature and Research Gap

The literature on loan default prediction highlights a clear trajectory from traditional statistical methods, such as logistic regression and linear discriminant analysis, to advanced machine learning approaches including Random Forest, XGBoost, and LightGBM. These newer models consistently demonstrate stronger predictive power, particularly in large and complex datasets, though often at the cost of reduced

interpretability (Couronné et al., 2018). Alongside model development, researchers have sought to address the challenges of class imbalance through resampling methods such as SMOTE and hybrid techniques, and have adopted a range of evaluation metrics, including recall, F1-score, and AUC-ROC, to provide more informative assessments.

One of the most significant problems in the literature is the use of post-loan features such as repayment history or delinquency updates after origination in model training, which can inflate reported performance but is not available at decision time.

Benchmarking work has also highlighted the need for fair, comparable evaluations in credit scoring (Simão, 2023). While such leakage produces impressively high accuracy scores in academic studies, these results are misleading and have no operational value in real-world lending. This explains why some published work reports near-perfect prediction results, whereas models built only on pre-loan application data, as in this dissertation, achieve more modest but far more realistic performance.

In addition, many studies place disproportionate emphasis on overall accuracy or AUC, rather than on recall, which more directly reflects the cost of missed defaults (Pandey et al., 2021). Finally, although interpretability methods such as SHAP and LIME have advanced significantly, the tension between transparency and predictive performance remains unresolved, leaving uncertainty about how lenders should balance these factors in practice (Rudin, 2019).

This dissertation addresses these gaps by focusing exclusively on pre-loan application data, evaluating multiple machine learning models across a broad set of metrics, and applying SHAP values to provide insights into feature contributions. By

explicitly avoiding post-loan data leakage and prioritising fairness in evaluation, this study contributes realistic and transparent benchmarks for loan default prediction. Few studies have combined all these aspects (imbalance handling, interpretability, leakage avoidance) in one pipeline, which is what this dissertation aims to do.

3. Methodology

This research adopts a quantitative, supervised machine learning approach to evaluate the performance and interpretability of models for predicting loan defaults using pre-loan application data from the Lending Club dataset. The methodology is designed to provide a fair comparison between traditional statistical methods and modern machine learning algorithms, with a focus on both predictive capability and transparency.

The models selected include Logistic Regression, representing traditional approaches, alongside Random Forest, XGBoost, and LightGBM as more advanced techniques. These models were chosen because they are widely applied in credit risk prediction and represent a balance between interpretability and performance (Lessmann et al., 2015). Evaluation goes beyond accuracy, incorporating precision, recall, F1-score, and AUC-ROC, which are more informative for imbalanced classification problems (Saito and Rehmsmeier, 2015).

3.1 Research Design

The study follows an experimental research design using secondary data from the Lending Club platform, covering loans issued between 2007 and 2018. The dataset is publicly available on Kaggle and contains approximately 2.26 million loan records with over 140 features.

The workflow of the research is structured as follows:

1. **Data acquisition and exploration** – inspecting dataset structure, understanding feature definitions, and examining the class distribution.
2. **Data preprocessing** – cleaning missing values, removing irrelevant or post-loan features, handling outliers, encoding categorical variables, scaling numerical features, and addressing multicollinearity.
3. **Class imbalance handling** – applying oversampling and hybrid resampling methods to balance the dataset.
4. **Model training** – implementing Logistic Regression, Random Forest, XGBoost, and LightGBM.
5. **Hyperparameter tuning** – optimising model performance using cross-validation and random search.
6. **Performance evaluation** – assessing predictive outcomes using multiple metrics, including AUC-ROC.
7. **Threshold analysis** – evaluating the effect of alternative classification thresholds.
8. **Interpretability analysis** – using feature importance rankings and SHAP values to explain model decisions.

This stepwise design ensures the study is transparent, reproducible, and aligned with best practices in credit risk modelling research (Brown and Mues, 2012).

After the initial workflow was implemented, baseline models were trained and evaluated to establish a reference point. These first results provided insight into the

challenges of class imbalance and highlighted which features and models carried the most predictive value. Following this, additional experiments were carried out to improve performance and robustness. These included enhanced feature engineering in R, cost-sensitive learning in ensemble models, and a simple stacking ensemble. Although stacking did not outperform the strongest individual models, these extended experiments ensured that the final results reflect both the baseline performance and the improvements achieved through a more advanced workflow. The attempted stacking model did not yield any performance gains (no F1 improvement over the best individual model), so results are not tabulated.

3.2 Dataset

The dataset is the publicly available Lending Club accepted loans file covering 2007–2018. Only loans with resolved outcomes are retained to enable supervised learning on a binary target (default vs non-default). Observations labelled as Current, In Grace Period, and Late categories are excluded. The target variable is defined as 1 for Charged Off / Default (including “Does not meet the credit policy. Status: Charged Off”) and 0 for Fully Paid (including “Does not meet the credit policy. Status: Fully Paid”).

To ensure the results are deployable at origination and to avoid data leakage, the model uses pre-loan application features only. Post-origination variables (such as repayments, recoveries, last payment dates, and delinquency updates after origination) are excluded. The dataset is class-imbalanced with non-defaults forming the majority. Consequently, evaluation focuses on recall, precision, F1-score and AUC rather than accuracy alone, as accuracy can be misleading under severe class imbalance (Sun et al., 2009).

$$fico_score_average = \frac{fico_range_low + fico_range_high}{2}$$

In addition to standard loan and borrower attributes used in credit risk modelling (loan amount, term, interest rate, annual income, DTI, employment length, home ownership, purpose, verification status, and revolving utilisation), new features were created to capture additional risk information. A stable proxy for creditworthiness was derived by averaging the reported low and high FICO ranges into a single variable, `fico_score_avg`. Credit-history aggregates available at application time — `open_acc`, `pub_rec`, and `total_acc` — were retained as they provide a summary of the applicant's credit profile. Temporal indicators (`issue_year`, `issue_month`) were extracted from the origination date to capture potential macroeconomic and cohort effects on loan performance. The variable `mths_since_last_delinq` was also included, with missing values set to a sentinel value of 999 to represent no delinquency on record. A binary flag was added alongside this to allow the models to distinguish between genuinely high values and missing entries.

3.3 Data Preprocessing

Data preprocessing was carried out to ensure that the dataset was clean, consistent, and suitable for modelling while avoiding the inclusion of information that would not be available at loan origination. First, columns with more than 95% missing values were removed, as such variables contribute little predictive power and risk introducing noise. Identifiers and all post-origination fields were also excluded to prevent data leakage, ensuring that only features available at the time of the lending decision were used. Median imputation was applied to numerical variables, as it is less sensitive to extreme values and has been widely used in credit scoring research (Little and Rubin, 2019).

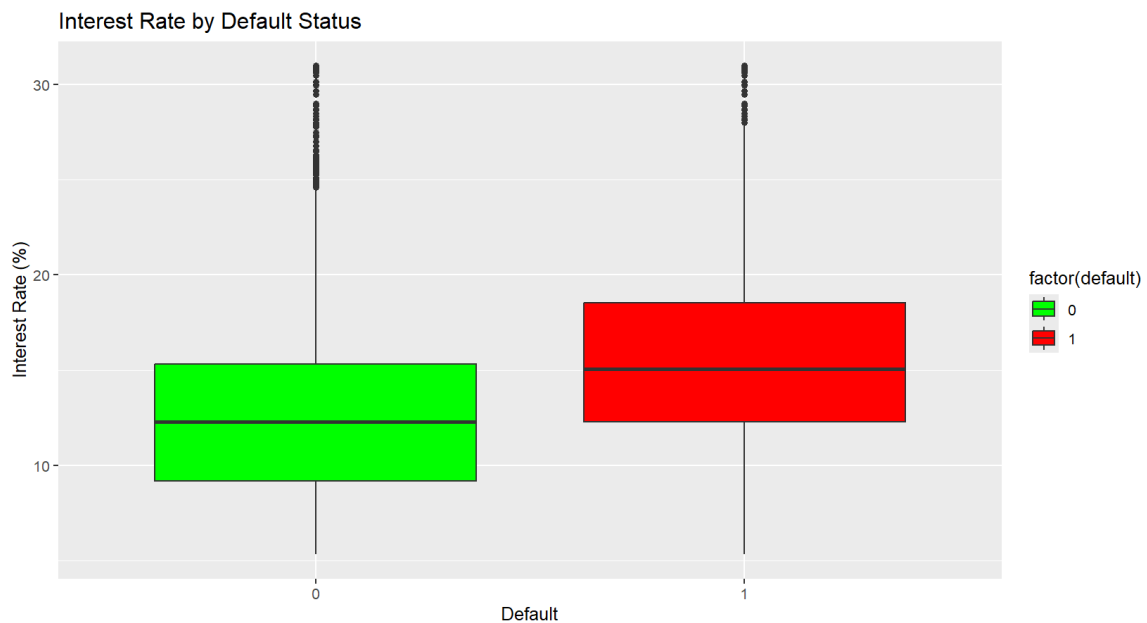


Figure 1 Interest Rate % vs Default

Several engineered variables were then created to strengthen predictive capacity. A stable way of measuring borrower creditworthiness was derived by averaging the reported FICO score ranges into a single feature, `fico_score_avg`. Historical indicators, `issue_year` and `issue_month`, were extracted from the origination date in order to capture potential macroeconomic and cohort effects on default risk. Credit history information was also retained through the inclusion of features such as `open_acc`, `pub_rec`, and `total_acc`, which provide a summary of the applicant's credit portfolio. The variable `mths_since_last_delinq` was treated with special care: missing values were imputed with a sentinel value of 999 to represent the absence of a recorded delinquency.

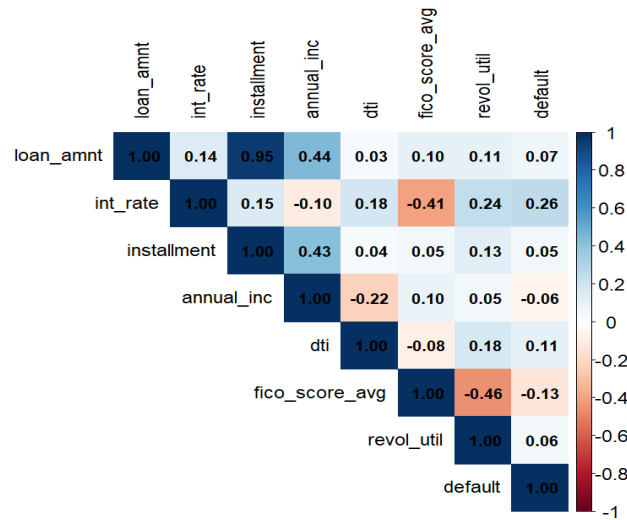


Figure 2 Correlation heatmap of features

To handle missing values in other variables, median imputation was applied to numerical fields (annual_inc, dti, revol_util, open_acc, pub_rec, total_acc), which avoids skewing the distribution compared to mean imputation. For the categorical variable emp_length, missing values were replaced with the label “Unknown” so that the feature could still be used without discarding records. Outliers were addressed to prevent extreme values from disproportionately influencing the models: annual income was capped at \$500,000, implausible debt-to-income ratios below 0 or above 100 were replaced with the sample median, and revolving utilisation was capped at 150%. A correlation analysis, as seen in *Figure 2*, showed loan_amnt and installment are highly collinear ($r \approx 0.95$). In the initial baseline preprocessing, we removed installment to simplify the feature space. In the enhanced preprocessing, we retained both variables: tree-based ensembles (Random Forest, XGBoost, LightGBM) are robust to multicollinearity, and the regularised Logistic Regression used here mitigates coefficient instability.

Moreover, installment carries a slightly different operational signal than loan_amnt alone. Re-introducing installment did not degrade performance, so it is kept in the final models. Categorical variables such as loan term, employment length, home ownership, purpose, and verification status were encoded using one-hot encoding to make them suitable for machine learning algorithms, while numerical features were standardised only when required to aid model convergence. The result of these steps was a clean, leakage-free dataset that balanced interpretability and predictive potential.

3.4 Class Imbalance Handling

A critical challenge in credit risk modelling is the imbalance between default and non-default cases. In the Lending Club dataset, the majority of loans are fully repaid, with defaults making up only a small proportion of the overall sample. This imbalance can bias machine learning algorithms toward predicting the majority class, producing high overall accuracy but very poor recall for defaulters. Such misclassifications are problematic because the financial cost of missing a default is typically much greater than the cost of incorrectly flagging a non-defaulting borrower (Sun et al., 2009).

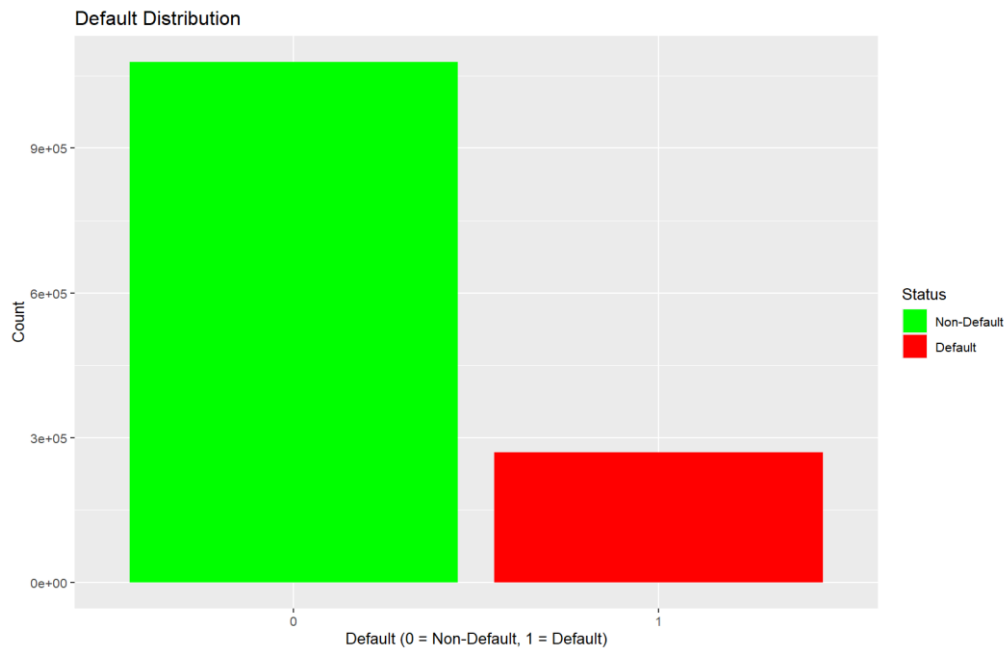


Figure 3 Imbalance between non defaults and defaults

To mitigate this issue, the dataset was rebalanced during model training. The primary method employed was the Synthetic Minority Over-sampling Technique (SMOTE), which generates synthetic minority samples by interpolating between existing default cases. Unlike simple duplication, SMOTE produces new, realistic observations that help the model better learn the characteristics of defaulters, improving sensitivity to the minority class.

Before balancing: default

0 755117

1 188552

Name: count, dtype: int64

After balancing: default

0 755117

1 755117

Name: count, dtype: int64

While SMOTE improves recall, it can also increase noise by creating synthetic cases near class boundaries as seen in the above snippet. To address this limitation, a hybrid approach was adopted in the form of SMOTE combined with Edited Nearest Neighbours (SMOTE-ENN). Hybrid resampling methods such as SMOTE-ENN have been shown to improve classifier robustness in financial datasets (Fernández et al., 2018). After oversampling, ENN removes noisy or borderline samples by checking class consistency with nearest neighbours. This additional filtering step helps refine the synthetic data and has been shown to improve both recall and F1-score in imbalanced classification problems.

In addition to these resampling approaches, cost-sensitive learning was also later incorporated into the modelling pipeline. Logistic Regression and Random Forest were trained with class weights that penalised the misclassification of defaulters more heavily than non-defaulters, while for XGBoost and LightGBM, the `scale_pos_weight` parameter was used to reflect the imbalance in class distribution. These adjustments allowed the algorithms themselves to account for the higher cost of false negatives and further improved their ability to detect defaulters.

The rebalancing strategies were applied only to the training set, ensuring that the test set remained representative of the natural class distribution. This approach prevents overly optimistic performance estimates and mirrors real-world lending scenarios, where defaulters remain the minority. By integrating both SMOTE and SMOTE-ENN, alongside cost-sensitive learning, into the modelling pipeline, this study ensures a robust comparison of models under conditions that reflect the true distribution of loan outcomes while also enhancing their ability to identify high-risk borrowers.

3.5 Model Selection and Training

This study evaluates four models: Logistic Regression, Random Forest, XGBoost, and LightGBM. These algorithms were selected because they are widely applied in credit risk prediction, over both traditional and modern approaches, and represent a balance between interpretability and predictive power. A simple stacking ensemble was also tested, but it did not outperform the best individual models and is therefore not emphasised in the results.

Logistic Regression (LR) was chosen as the baseline model, given its long-standing use in credit scoring. Logistic Regression estimates the probability of default as a linear function of borrower characteristics, producing coefficients that are straightforward to interpret. Although limited in its ability to capture non-linear interactions, LR remains the most widely implemented method in industry due to its transparency and regulatory acceptance (Thomas, 2009).

Random Forest (RF) was included as an ensemble method that addresses the instability of single decision trees by aggregating the predictions of many trees trained on bootstrap samples. By averaging across multiple trees, Random Forest reduces variance and improves generalisation, while providing feature importance measures that can help identify key drivers of default. Research in peer-to-peer lending shows that Random Forest often outperforms Logistic Regression in predictive accuracy, particularly in noisy, high-dimensional datasets (Malekipirbazari & Aksakalli, 2015).

XGBoost (Extreme Gradient Boosting) represents a state-of-the-art boosting approach. Unlike bagging methods, boosting builds trees sequentially, with each new tree correcting the errors of the previous ensemble. XGBoost has become one of the

most widely used algorithms in credit risk research because of its scalability, speed, and strong predictive performance. Empirical studies show that XGBoost frequently surpasses both Random Forest and Logistic Regression in default prediction tasks (Yang et al., 2025).

LightGBM, developed by Microsoft, extends the gradient boosting framework with histogram-based learning and leaf-wise tree growth, which significantly improves computational efficiency. LightGBM can train faster and use less memory than XGBoost while achieving similar levels of predictive performance, making it especially suitable for large-scale financial datasets. Studies applying LightGBM to credit scoring demonstrate that it offers both efficiency and strong classification accuracy in high-dimensional settings (Ke et al., 2017).

By comparing these four models, the study captures both the interpretability of traditional methods and the predictive strength of advanced ensemble algorithms, providing a realistic evaluation of their aptness for loan default prediction.

3.6 Hyperparameter Tuning

Optimising model performance requires careful selection of hyperparameters, which control how algorithms learn from data. Unlike model parameters, which are learned during training, hyperparameters must be defined prior to training and can significantly affect both predictive accuracy and generalisation. To ensure a fair comparison across models, systematic hyperparameter tuning was applied to Logistic Regression, Random Forest, XGBoost, and LightGBM.

For Logistic Regression, tuning focused on the choice of regularisation technique and the strength of the penalty term. Regularisation mitigates overfitting by restricting model coefficients, which is particularly important in datasets with correlated

features. The solver was also optimised to ensure stable convergence given the size of the dataset.

For Random Forest, key hyperparameters included the number of trees, maximum tree depth, and the minimum number of samples required to split a node. Increasing the number of trees generally improves stability, while controlling tree depth prevents overfitting. Tuning these values ensures a balance between predictive accuracy and computational efficiency.

For XGBoost and LightGBM, tuning focused on learning rate, maximum depth, and the number of boosting rounds, along with regularisation parameters. The learning rate determines how quickly the algorithm updates during training, while tree depth influences model complexity. These boosting algorithms are sensitive to hyperparameter settings, making systematic optimisation essential for achieving strong performance.

The tuning process employed Randomised Search with Cross-Validation (Random Search CV). Random Search was selected over Grid Search because it is more computationally efficient when exploring high-dimensional hyperparameter spaces, while still offering strong performance (Bergstra and Bengio, 2012). Stratified five-fold cross-validation was used to preserve the proportion of defaults and non-defaults in each fold, ensuring that class imbalance did not distort validation results. The best hyperparameter configuration for each model was then retrained on the full training set before final evaluation on the hold-out test set.

This approach ensured that each model was optimised fairly and consistently, enabling a meaningful comparison of their predictive performance and interpretability.

3.7 Performance Evaluation

Evaluating model performance in loan default prediction requires the use of metrics that account for the imbalanced nature of the dataset. Accuracy alone is insufficient, as a model could achieve high accuracy by predicting nearly all cases as non-defaults while failing to identify actual defaulters. To address this, a combination of complementary evaluation metrics was employed.

Accuracy was retained as a baseline measure, representing the proportion of correctly classified loans across both classes. However, it was not used in isolation due to its tendency to obscure poor performance on minority cases (Bekhet and Eletter, 2014).

Precision was calculated as the proportion of loans predicted to default that actually defaulted. This metric reflects the cost of false positives, which in credit scoring corresponds to rejecting creditworthy applicants. High precision therefore indicates a conservative model that minimises unnecessary credit denials (Pandey et al., 2021).

Recall, or sensitivity, was prioritised as a key performance metric, as it measures the proportion of actual defaulters correctly identified. In lending contexts, missing a defaulter (false negative) typically has more severe financial consequences than incorrectly flagging a non-defaulter (false positive). Models with higher recall are therefore more valuable for risk management (Sun et al., 2009).

To balance the trade-off between precision and recall, the F1-score was used as a mean of the two. This single value provides a more complete assessment of model performance on imbalanced data, ensuring that neither precision nor recall is optimised at the expense of the other (Ali et al., 2015).

Finally, the Area Under the Receiver Operating Characteristic Curve (AUC-ROC) was included as a threshold-independent metric. AUC-ROC measures the ability of a model to discriminate between defaulters and non-defaulters across all possible classification thresholds. A higher AUC indicates stronger ranking capability, making it particularly useful when comparing models of differing complexity (Saito and Rehmsmeier, 2015).

By combining these metrics, the study provides a balanced and fair evaluation of all models, ensuring that both predictive performance and practical business implications are accounted for.

3.8 Threshold Analysis

In binary classification tasks, models typically use a default probability threshold of 0.5 to assign class labels. However, in imbalanced datasets such as loan default prediction, this threshold may not be optimal. A high threshold can result in very few predicted defaults, yielding high precision but poor recall, while a low threshold increases recall at the cost of more false positives. Given the financial implications of missed defaults, threshold selection becomes an important step in model evaluation.

To explore this trade-off, the study implemented threshold analysis by varying the classification cut-off and recording changes in accuracy, precision, recall, and F1-score. This approach highlights how different operating points align with different risk preferences. For example, a lower threshold may be more suitable in contexts where lenders aim to minimise financial losses from undetected defaults, even if it means rejecting more creditworthy applicants. Conversely, a higher threshold might be preferred where lenders prioritise maintaining approval rates.

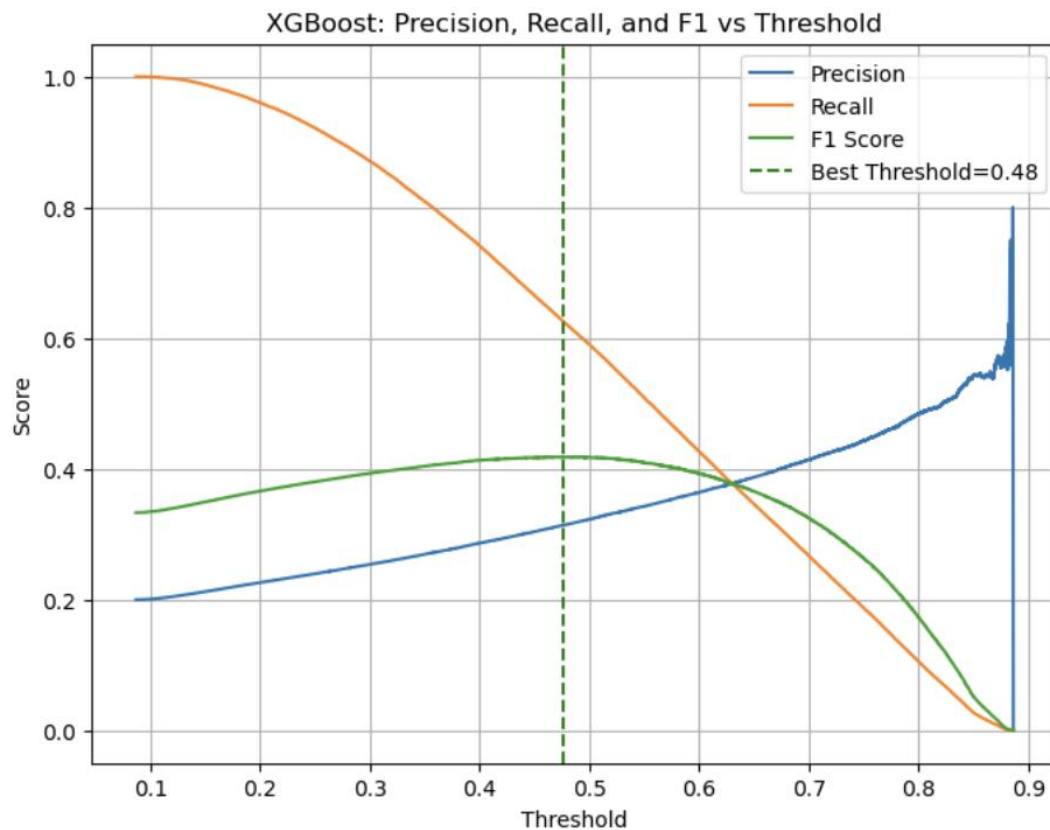


Figure 4 Precision vs Recall with different thresholds

The Precision–Recall curve provides a useful visual tool for this process, since it shows the relationship between precision and recall across thresholds, as seen in Figure 4. By examining this curve, the most appropriate threshold can be chosen based on the business objective—whether to maximise recall, precision, or a balance reflected by the F1-score (Saito and Rehmsmeier, 2015).

In this study, thresholds of 0.25, 0.30, 0.35, and 0.40 were tested alongside the default 0.50 cut-off. Results were compared across all metrics to evaluate trade-offs and identify thresholds that provided improved recall and F1-score without disproportionately sacrificing precision. This analysis ensured that model performance was assessed not only at the conventional cut-off but also at alternative decision points that better reflect the cost-sensitive nature of lending decisions.

3.9 Model Interpretability

In addition to predictive performance, model interpretability is a critical consideration in loan default prediction. Credit decisions affect individuals and institutions directly, and in regulated financial environments, models must not only be accurate but also explainable. Without interpretability, high-performing models may be unsuitable for deployment, as lenders must demonstrate why an applicant was accepted or rejected (Rudin, 2019).

Interpretability was assessed using two complementary approaches: feature importance and SHAP values. Feature importance was derived from tree-based models such as Random Forest, XGBoost, and LightGBM. This method quantifies the contribution of each variable to the model's predictions by measuring how much a feature reduces impurity when used for splitting in decision trees. Features with higher importance scores are deemed more influential in classification outcomes. While feature importance provides a global view of the most relevant predictors, it does not explain individual-level decisions (Lundberg and Lee, 2017).

To address this limitation, SHAP (Shapley Additive Explanations) was applied. SHAP is a game-theoretic approach that assigns each feature a contribution value for individual predictions. This allows the decomposition of a single prediction into feature-level impacts, showing which variables pushed the model towards predicting default or non-default. SHAP has been widely recognised as one of the most consistent and theoretically sound methods for interpreting complex models, as it satisfies desirable properties such as local accuracy and additivity (Lundberg and Lee, 2017).

In this study, SHAP was applied not only to XGBoost but also to LightGBM, which achieved the strongest overall performance. Applying SHAP in this way provided both global insights by averaging feature contributions across the dataset to highlight the most influential variables overall and local insights, by explaining specific loan-level predictions. This dual perspective is especially valuable in financial applications: global interpretations help institutions design fairer credit policies, while local explanations support transparency in individual lending decisions. Post-hoc interpretability techniques such as SHAP and LIME are increasingly used in financial services to satisfy regulatory requirements (Molnar, 2020)

By combining feature importance with SHAP analysis, this research ensures that the trade-off between predictive power and transparency is critically examined. While ensemble models such as XGBoost and LightGBM deliver superior performance, the use of interpretability techniques allows their predictions to be unpacked and understood, aligning the methodology with the regulatory and ethical standards required in credit risk management.

4. Results

This chapter presents the results of the modelling process and evaluates the performance of the selected algorithms. The analysis is structured to reflect the development of the work. Initial results were obtained using the baseline models and resampling methods. These results provided a benchmark against which further improvements could be measured. Additional experiments were then conducted, which introduced enhanced feature engineering, cost-sensitive learning, and interpretability analysis.

4.1 Baseline Model Performance

The baseline models of Logistic Regression and Random Forest were first applied to the cleaned dataset without any resampling. Logistic Regression, trained with standardisation and balanced class weights, achieved an F1-score of 0.42 with recall of 0.63 and precision of 0.32. This showed that even a simple linear model was able to detect a significant proportion of defaults, though with modest precision. A reduced feature set was also tested based on Random Forest importance ranks, and found no significant drop in performance, confirming that the model was not overfitting to noise features. In contrast, the default Random Forest achieved high accuracy of 0.80 but recall was only 0.06, resulting in an F1-score of 0.11. This indicated that without adjustment for imbalance, Random Forest was biased heavily toward predicting the majority class, capturing very few defaulters.

These baseline results confirmed the challenge of class imbalance. High accuracy was misleading, as it did not reflect the model's inability to identify defaults. Logistic Regression provided a more balanced outcome, but further work was needed to improve recall without sacrificing interpretability.

Model	Precision	Recall	F1-score	AUC
Logistic Regression (balanced)	0.32	0.63	0.42	~0.65
Random Forest (default)	0.52	0.06	0.11	~0.80
Random Forest (SMOTE-ENN @ 0.25)	0.28	0.78	0.41	~0.70
XGBoost (SMOTE-ENN tuned @ 0.48)	0.31	0.63	0.42	~0.71

Model	Precision	Recall	F1-score	AUC
LightGBM (SMOTE-ENN tuned)	0.31	0.63	0.42	~0.72

Figure 5 Initial results of ML models

4.2 Resampling with SMOTE and SMOTE-ENN

To address the imbalance, SMOTE and SMOTE-ENN were applied to the training data. With SMOTE, Random Forest performance improved slightly, with recall increasing to 0.15 and F1-score to 0.22. However, the improvement was limited. The hybrid SMOTE-ENN method proved more effective. By combining oversampling with noise reduction, it produced recall values above 0.60 when thresholds were adjusted.

Threshold analysis was conducted by lowering the classification cut-off from 0.50 to values between 0.40 and 0.25. At a threshold of 0.25, Random Forest with SMOTE-ENN achieved recall of 0.78 and F1-score of 0.41, representing a substantial gain in sensitivity to defaults. This came at the cost of precision and overall accuracy, but the results aligned with the business context, where detecting more defaulters is often preferable to avoiding false positives.

4.3 Gradient Boosting Models

Boosting algorithms were then introduced to provide a more advanced benchmark. Both XGBoost and LightGBM achieved F1-scores of around 0.42 when trained with SMOTE-ENN and threshold tuning. Recall and precision were balanced at approximately 0.63 and 0.31 respectively, showing that boosting methods could achieve similar performance to Random Forest with less reliance on aggressive threshold adjustments.

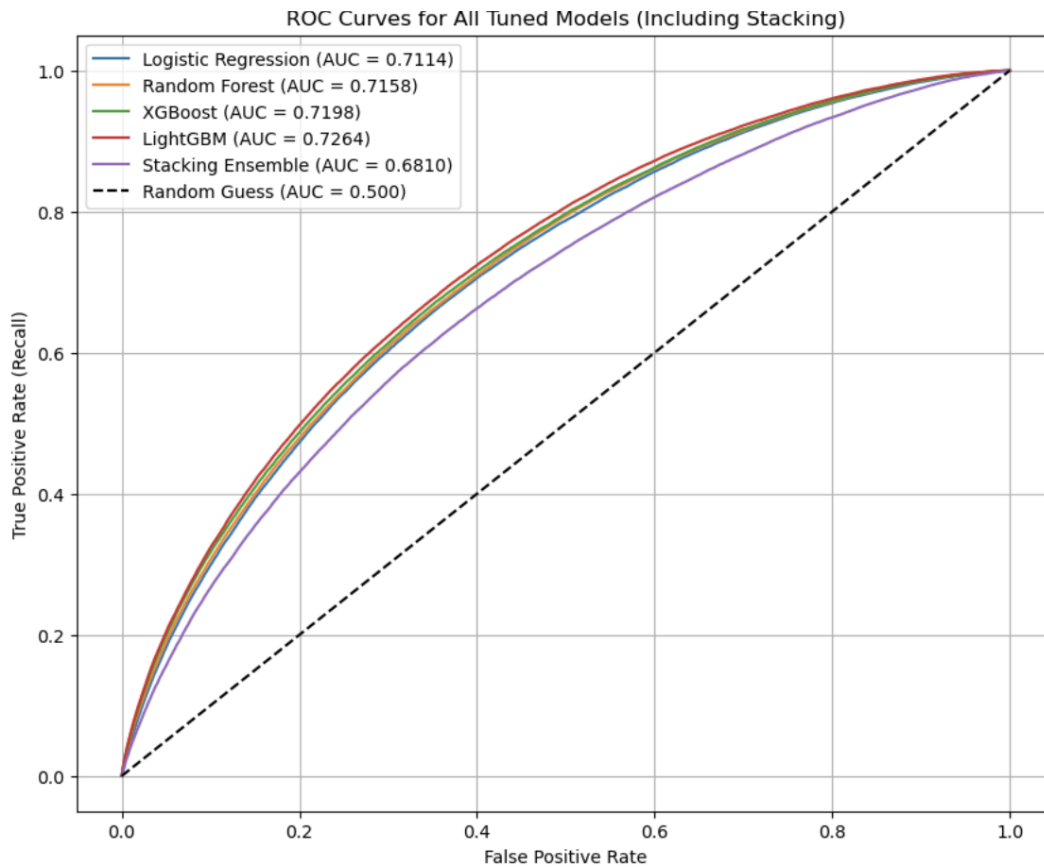


Figure 6 ROC curves on the test set for top models; LightGBM achieves the highest AUC

Principal Component Analysis (PCA) was also tested as a dimensionality reduction technique. However, the results showed no significant improvement in F1-score or recall compared to models trained on the full feature set. This suggested that PCA was not necessary for these algorithms, which already handle high-dimensional data effectively.

4.4 Enhanced Feature Engineering and Cost-Sensitive Learning

Building on the initial experiments, additional preprocessing was performed using R to engineer new features and strengthen the dataset. These included temporal indicators (issue_year, issue_month), credit history aggregates (open_acc, pub_rec, total_acc), and a sentinel encoding of missing values in mths_since_last_delinq accompanied by a binary missingness flag. As seen in Figure 7, the default rate was

higher during periods of global financial instability. Factors as such make the task of predicting loan defaults a lot more complex since these trends are harder to capture.

These features were incorporated into the enhanced dataset to capture more information available at loan origination. Unlike the baseline preprocessing where installment was removed, the enhanced pipeline reintroduced installment alongside loan_amnt. No adverse effect was observed on stability or performance, and it was retained in the final LightGBM model.

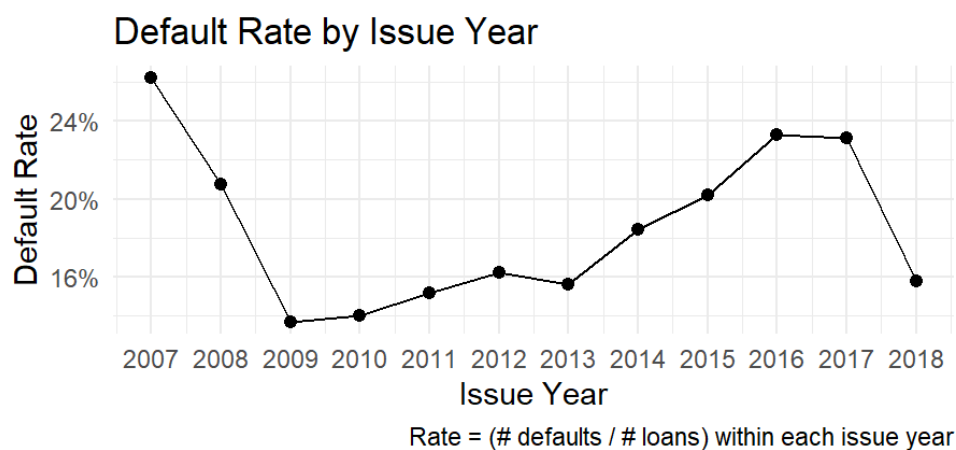


Figure 7 Default Rate vs Loan Issue Year

Alongside this, cost-sensitive learning was introduced. Logistic Regression and Random Forest were trained with class weights, while XGBoost and LightGBM used the scale_pos_weight parameter. This approach placed greater penalty on misclassifying defaulters, making the algorithms more responsive to the minority class.

Model	Precision	Recall	F1-score	AUC
Logistic Regression (class-weight)	~0.33	~0.61	0.41	~0.68
Random Forest (class-weight)	~0.32	~0.65	0.42	~0.71
XGBoost (scale_pos_weight)	~0.32	~0.62	0.42	~0.72
LightGBM (enhanced + scale_pos_weight)	0.36	0.59	0.44	0.73

Figure 8 Final ML model scores

The enhanced dataset combined with cost-sensitive boosting produced the strongest results. LightGBM in particular achieved an F1-score of approximately 0.44 and an AUC of 0.73. While the improvement in F1-score over earlier models was modest, the higher AUC demonstrated stronger ranking ability, meaning the model was better at ordering borrowers by risk even when a fixed threshold was applied.

4.5 Model Interpretability

Interpretability analysis was performed using both feature importance measures and SHAP values. Across all tree-based models, interest rate, average FICO score, and loan term consistently emerged as top predictors. With the enhanced dataset, temporal features such as issue_year also became influential, reflecting the role of macroeconomic and cohort effects in default risk.

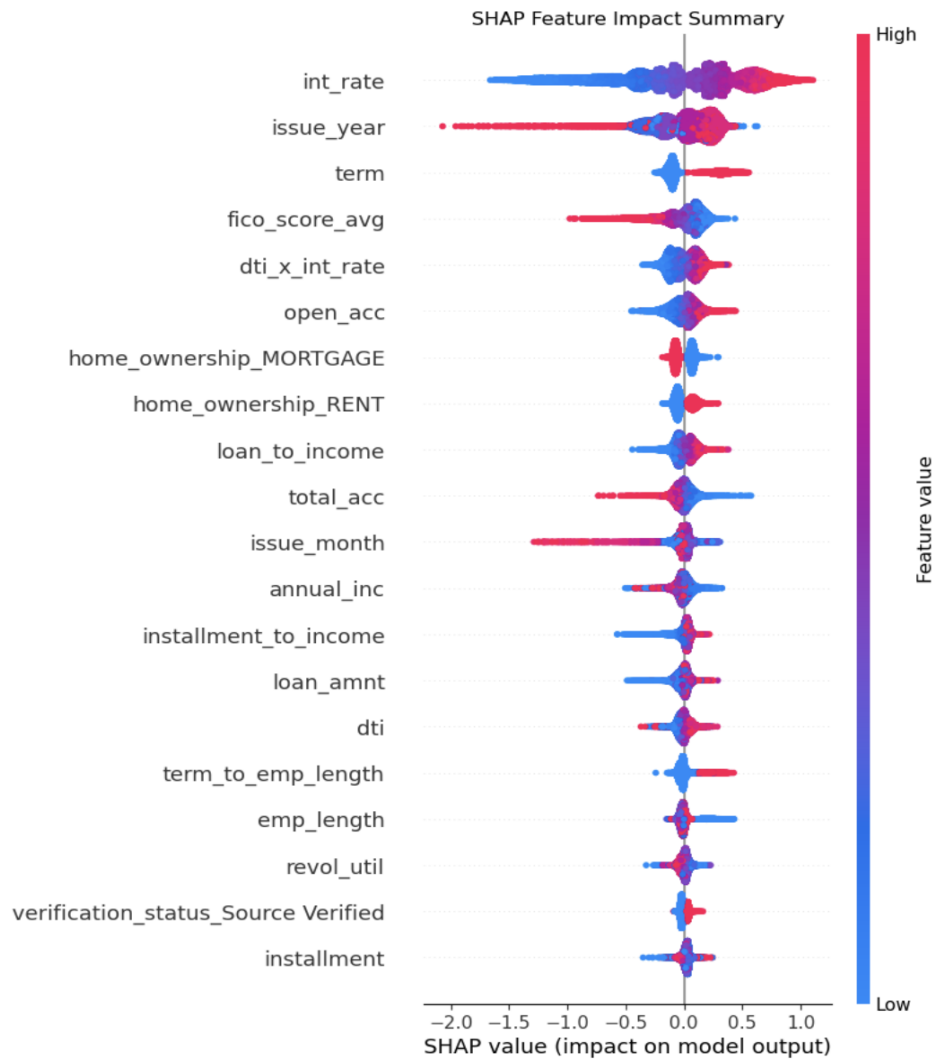


Figure 9. SHAP values for interpretability ranked from most to least impactful

SHAP values, as seen in Figure 9, provided both global and local explanations of model behaviour. A negative SHAP value indicates the model chose non default, and a positive value indicates more likely to default. The global summary plots showed that higher interest rates and lower FICO scores pushed predictions towards default, while higher incomes and verified applications reduced default risk. Local explanations allowed individual predictions to be decomposed into feature contributions, offering transparency at the case level. This interpretability supported the regulatory requirement for explainable credit models while also validating that the model was learning sensible relationships.

4.6 Summary of Results

The results show a clear progression. Logistic Regression established a strong baseline under imbalance, while Random Forest required resampling and threshold tuning to achieve useful recall. SMOTE-ENN combined with threshold adjustment significantly improved recall, though at the expense of precision. Boosting methods such as XGBoost and LightGBM matched these results more efficiently, and the addition of enhanced feature engineering and cost-sensitive learning further improved LightGBM to $F1 \approx 0.44$ and $AUC \approx 0.73$.

Overall, the models demonstrated that realistic pre-loan features can achieve moderate but meaningful predictive performance. Importantly, results avoided the inflation seen in studies that include post-loan information. The combination of boosting, enhanced features, cost sensitivity, and SHAP explanations provided a balanced outcome between predictive strength and interpretability.

5. Discussion

This chapter interprets the results presented in Chapter 4 in the context of existing literature on loan default prediction. It evaluates how the models performed relative to expectations, examines the trade-offs between predictive accuracy and interpretability, and reflects on the implications for credit risk modelling in practice.

5.1 Model Performance and Class Imbalance

The results confirmed that class imbalance is a central obstacle in loan default prediction. Logistic Regression, despite its simplicity, achieved a stronger balance of recall and F1-score than the default Random Forest, which initially performed poorly on defaulters. This finding aligns with research showing that traditional statistical

models, while limited in flexibility, can still provide stable baselines in highly imbalanced settings (Thomas, 2009). Random Forest required resampling and threshold adjustment before it was able to meaningfully detect defaults, illustrating the importance of applying imbalance handling techniques in practice.

SMOTE-ENN combined with threshold tuning proved effective in recovering recall, pushing it as high as 0.78 at a cut-off of 0.25, although precision and overall accuracy fell as a result. This outcome supports the argument that recall must be prioritised in credit risk contexts, as the financial cost of missed defaults is usually greater than that of false positives (Sun et al., 2009). The trade-off observed in this study demonstrates that lenders may need to accept higher rejection rates of creditworthy borrowers in order to reduce losses from undetected defaulters.

5.2 Boosting Methods and Comparative Findings

Boosting algorithms, specifically XGBoost and LightGBM, achieved balanced recall and precision without requiring extreme threshold adjustments. Their F1-scores were similar to Random Forest with resampling but were achieved more efficiently, confirming previous research that boosting methods consistently outperform or match traditional models in credit risk prediction (Brown and Mues, 2012).

The strongest overall performance came from LightGBM when combined with enhanced feature engineering and cost-sensitive learning. Although the improvement in F1-score compared to earlier experiments was modest, the AUC of 0.73 indicated stronger ranking capability. This supports findings in the literature that boosting methods are well-suited to complex, high-dimensional datasets and provide lenders with valuable discriminatory power, even when absolute recall remains difficult to maximise.

5.3 Feature Engineering and Temporal Effects

The enhanced preprocessing in R added features such as temporal indicators and credit history aggregates, which improved model robustness. Notably, the inclusion of `issue_year` as a predictor reflected macroeconomic or cohort effects that influenced default risk. This finding is consistent with evidence that credit risk is not static but changes over time with economic conditions (Malekipirbazari and Aksakalli, 2015). While the overall gains in F1-score were moderate, the improvement in AUC showed that engineered features strengthened the model's ability to rank borrowers, providing more flexibility for lenders to set thresholds according to business needs.

5.4 Interpretability and Practical Implications

Interpretability was achieved through feature importance measures and SHAP values. The results consistently showed that interest rate, FICO score, and loan term were among the most important predictors, aligning with domain knowledge and confirming that the models were learning sensible relationships. The addition of temporal features further highlighted how defaults varied across issue years, giving insight into shifts in borrower risk. SHAP also enabled case-level explanations, showing which features contributed most to individual predictions.

These findings reinforce the importance of interpretability in regulated industries (Barredo Arrieta et al., 2020). While ensemble methods such as LightGBM provided stronger performance, SHAP analysis ensured that their predictions remained transparent and auditable. Explanation tools are widely used to make complex models more transparent in practice (Molnar, 2020). At the same time, some scholars argue that for high-stakes decisions, inherently interpretable models should be preferred over black-box models with explanations (Rudin, 2019).

5.5 Realistic Benchmarks and Limitations

A key contribution of this study is that all models were trained exclusively on pre-loan application features. This avoided the leakage seen in many academic studies, where post-loan variables such as repayment behaviour are used in training. As noted in prior work, including such features can artificially inflate performance but has little real-world value at loan origination (Simão, 2023). The more modest results in this study, with F1-scores around 0.44 and AUC up to 0.73, therefore represent realistic and deployable benchmarks.

Nevertheless, limitations remain. The dataset was split randomly into training and testing sets, which means temporal generalisation was not directly tested. In practice, lenders often apply models trained on earlier cohorts to later loans, so performance may differ over time. Future research could address this by applying rolling-window validation to better capture temporal drift. In addition, while SHAP provided valuable explanations, interpretability remains a challenge when dealing with complex feature interactions, and inherently interpretable models may still be preferable in some regulatory contexts.

6. Conclusion

This dissertation set out to develop and evaluate a machine learning pipeline for predicting loan defaults using the publicly available Lending Club dataset. The study focused on pre-loan application features to ensure that models remained deployable at the point of credit origination and avoided data leakage. The research aim was to provide a fair comparison between traditional and advanced algorithms, addressing the challenges of class imbalance while ensuring transparency through interpretability methods.

The results demonstrated that Logistic Regression established a strong baseline, particularly when class weights were applied. Random Forest required resampling and threshold tuning before it was able to capture defaulters effectively, highlighting the limitations of default ensemble settings on imbalanced data. SMOTE-ENN combined with threshold adjustments produced significant gains in recall, but at the cost of reduced precision and accuracy. Boosting methods, specifically XGBoost and LightGBM, offered stronger overall performance, achieving balanced recall and precision without the need for extreme threshold shifts. LightGBM, trained on the enhanced feature set and using cost-sensitive learning, achieved the best performance with an F1-score of 0.44 and an AUC of 0.73.

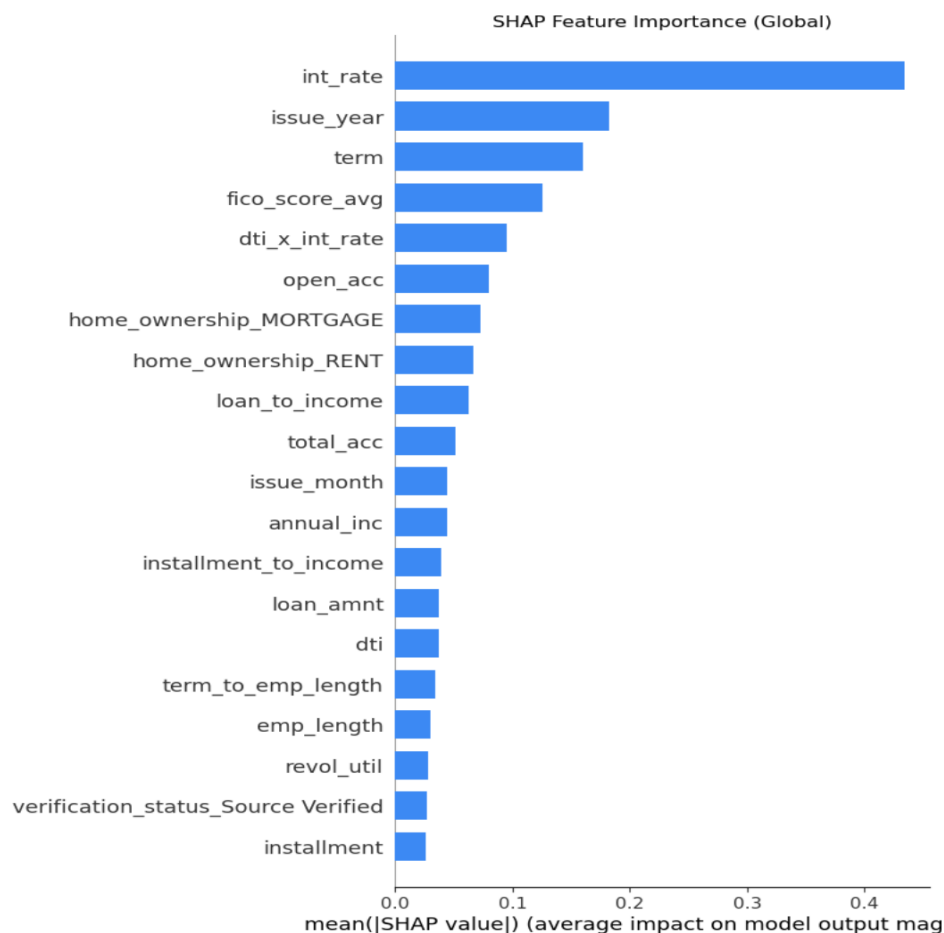


Figure 10 SHAP most important features

The analysis confirmed (Fig. 10) that interest rate, FICO score, and loan term were consistently the most influential features, with sequential variables such as issue year also contributing meaningfully to the enhanced dataset. SHAP provided both global and local explanations, ensuring that the models' decisions could be understood at both the population and individual level. This interpretability is essential for aligning advanced machine learning models with the transparency requirements of regulated financial industries (Rudin, 2019).

The study makes three main contributions. First, it establishes realistic performance benchmarks for loan default prediction using only pre-loan features, avoiding the inflated results seen in studies that rely on post-loan variables (Simão, 2023). Second, it shows that combining feature engineering, cost-sensitive boosting, and threshold analysis can improve recall and ranking ability in imbalanced settings, while still maintaining interpretability. Third, it demonstrates that explanation tools such as SHAP can bridge the gap between predictive performance and regulatory accountability.

Several limitations remain. The dataset split was random rather than chronological, meaning that changes in borrower behaviour over time were not fully captured. Future work could address this by applying rolling-window validation or retraining models on newer cohorts to test temporal robustness. In addition, while SHAP provided valuable post-hoc explanations, interpretability in credit risk modelling continues to face trade-offs, and fundamentally transparent models remain an alternative worth exploring. Finally, this study focused on a single dataset, and replication across other lending platforms would help confirm the generalisability of the findings.

In conclusion, this dissertation shows that machine learning can provide meaningful improvements in loan default prediction when combined with careful preprocessing, class imbalance handling, and interpretability techniques. While the results remain modest compared to studies that incorporate post-loan leakage, they are realistic, deployable, and aligned with the requirements of practical credit decision-making.

7. References

1. **Abellán, J. and Castellano, J.G. (2017).** *A comparative study on base classifiers in ensemble methods for credit scoring.* Expert Systems with Applications, 73, pp.1–10. <https://doi.org/10.1016/j.eswa.2016.12.020>
2. **Ali, A., Shamsuddin, S.M. and Ralescu, A.L. (2015).** *Class imbalance problem: A review.* International Journal of Advanced Soft Computing and Applications, 7(3), pp.176–204.
3. **Alvi, Jahanzaib & Arif, Imtiaz & Nizam, Kehkashan. (2024).** *Advancing Financial Resilience: A Systematic Review of Default Prediction Models and Future Directions in Credit Risk Management.* [10.1016/j.heliyon.2024.e39770](https://doi.org/10.1016/j.heliyon.2024.e39770)
4. **Bank for International Settlements (2019).** *How do machine learning and non-traditional data affect credit scoring? New evidence from a Chinese fintech firm.* BIS Working Papers No. 834. Available at: <https://www.bis.org/publ/work834.pdf>
5. **Barredo Arrieta, A., Díaz-Rodríguez, N., Del Ser, J. et al. (2020).** *Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI.* Information Fusion, 58, pp.82–115. <https://arxiv.org/pdf/1910.10045v1>

6. **Batista, G.E.A.P.A., Prati, R.C. and Monard, M.C. (2004).** *A study of the behaviour of several methods for balancing machine learning training data.* ACM SIGKDD Explorations Newsletter, 6(1), pp.20–29.
<https://doi.org/10.1145/1007730.100773>
7. **Bekhet, H.A. and Eletter, S.F.K. (2014).** *Credit risk assessment model for Jordanian commercial banks: Neural scoring approach.* Review of Development Finance, 4(1), pp.20–28. DOI: [10.1016/j.rdf.2014.03.002](https://doi.org/10.1016/j.rdf.2014.03.002)
8. **Bergstra, J. and Bengio, Y. (2012).** *Random search for hyper-parameter optimization.* Journal of Machine Learning Research, 13, pp.281–305.
jmlr.org/papers/volume13/bergstra12a
9. **Brown, I. and Mues, C. (2012).** *An experimental comparison of classification algorithms for imbalanced credit scoring data sets.* Expert Systems with Applications, 39(3), pp.3446–3453. DOI: [10.1016/j.eswa.2011.09.033](https://doi.org/10.1016/j.eswa.2011.09.033)
10. **Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002).** *SMOTE: Synthetic minority over-sampling technique.* Journal of Artificial Intelligence Research, 16, pp.321–357.
<https://doi.org/10.48550/arXiv.1106.1813>

11. **Couronné, R., Probst, P. & Boulesteix (2018).**, *AL. Random forest versus logistic regression: a large-scale benchmark experiment. BMC Bioinformatics* **19**, 270. DOI: <https://doi.org/10.1186/s12859-018-2264-5>
12. **Fernández, A., García, S., Galar, M., Prati, R.C., Krawczyk, B. and Herrera, F. (2018).** *Learning from Imbalanced Data Sets*. Cham: Springer.
[DOI: 10.1016/j.ins.2013.07.007](https://doi.org/10.1016/j.ins.2013.07.007)
13. **Goodman, B. and Flaxman, S. (2017).** *European Union regulations on algorithmic decision-making and a “right to explanation”*. *AI Magazine*, 38(3), pp.50–57. DOI: <https://doi.org/10.1609/aimag.v38i3.2741>
14. **He, H. and Garcia, E.A. (2009).** *Learning from imbalanced data*. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), pp.1263–1285.
DOI: [10.1109/TKDE.2008.239](https://doi.org/10.1109/TKDE.2008.239)
15. **Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. and Liu, T.-Y. (2017).** *LightGBM: A highly efficient gradient boosting decision tree*. *Advances in Neural Information Processing Systems*, 30. Accessed through: papers.nips.cc/paper_files/paper

16. **Lessmann, S., Baesens, B., Seow, H.V. and Thomas, L.C. (2015).** *Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research.* European Journal of Operational Research, 247(1), pp.124–136. <https://doi.org/10.1016/j.ejor.2015.05.030>
17. **Little, R.J.A. and Rubin, D.B. (2019).** *Statistical Analysis with Missing Data.* 3rd ed. Hoboken, NJ: Wiley. DOI: 10.1002/9781119482260
18. **Lundberg, S.M. and Lee, S.I. (2017).** *A unified approach to interpreting model predictions.* Advances in Neural Information Processing Systems, 30, pp.4765–4774. DOI: 10.48550/ARXIV.1705.07874
19. **Malekipirbazari, M. and Aksakalli, V. (2015).** *Risk assessment in social lending via random forests.* Expert Systems with Applications, 42(10), pp.4621–4631. DOI: 10.1016/j.eswa.2015.02.001
20. **Mirzamomen, Z. and Kangavari, MR. (2017).** *A framework to induce more stable decision trees for pattern classification, Pattern Analysis and Applications* , vol. 20, no. 4, pp. 991-1004. <https://doi.org/10.1007/s10044-016-0542-2>
21. **Molnar, C. (2020).** *Interpretable Machine Learning.* 2nd ed. Munich: Leanpub. <https://christophm.github.io/interpretable-ml-book>

22. **Monje, L., Carrasco, R.A. and Sánchez-Montañés, M. (2025).** *Machine learning XAI for early loan default prediction. Computational Economics.* DOI: 10.1007/s10614-025-10962-9
23. **Óskarsdóttir, M., Bravo, C., Sarraute, C., Vanthienen, J. and Baesens, B. (2019).** *The value of big data for credit scoring: Enhancing financial inclusion using mobile phone data and social network analytics.* Applied Soft Computing, 74, pp.26–39. DOI: **10.1016/j.asoc.2018.10.037**
24. **Pandey, M.K., Mittal, M. and Subbiah, K. (2021).** *Optimal balancing and efficient feature ranking approach to minimize credit risk.* International Journal of Information Management Data Insights, 1(2), 100037. DOI: 10.1016/j.jjime.2021.100037
25. **Ribeiro, M.T., Singh, S. and Guestrin, C. (2016).** *“Why should I trust you?”: Explaining the predictions of any classifier.* In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp.1135–1144. DOI: 10.1145/2939672.2939778
26. **Rudin, C. (2019).** *Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead.* Nature Machine Intelligence, 1(5), pp.206–215. DOI: 10.1038/s42256-019-0048-x

27. **Saito, T. and Rehmsmeier, M. (2015).** *The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets.* PLOS ONE, 10(3), p.e0118432. DOI: 10.1371/journal.pone.0118432
28. **Simão, S.B.S. (2023).** *Machine learning applied to credit risk assessment: Prediction of loan defaults.* MSc Dissertation, NOVA IMS, Universidade Nova de Lisboa. <http://hdl.handle.net/10362/149818>.
29. **Sun, Y., Wong, A.K.C. and Kamel, M.S. (2009).** *Classification of imbalanced data: A review.* International Journal of Pattern Recognition and Artificial Intelligence, 23(4), pp.687–719. DOI: 10.1142/S0218001409007326
30. **Thomas, L.C. (2009).** *Consumer Credit Models: Pricing, Profit, and Portfolios.* Oxford: Oxford University Press. ISBN: 978-0199232130
31. **Thomas, L.C., Crook, J.N. and Edelman, D.B. (2017).** *Credit Scoring and Its Applications.* 2nd ed. Philadelphia: SIAM. DOI: 10.1137/1.9781611974560

32. **Yang, S., Huang, Z., Xiao, W., Shen, X. (2025).** *Interpretable Credit Default Prediction with Ensemble Learning and SHAP*. DOI:
<https://doi.org/10.48550/arXiv.2505.20815>

Appendix

Appendix A – R Code for Exploratory Data Analysis

```
library(tidyverse)
library(ggplot2)
library(corrplot)

accepted_2007_to_2018Q4 <- read_csv("accepted_2007_to_2018q4.csv/accepted_2007_to_2018Q4.csv")
accepted_2007_to_2018Q4

lc_data_def <- read_csv("Lending Club Data Dictionary Approved.csv")
lc_data_def

lc_data_def %>%select(`LoanStatNew`, Description) %>% print(n = 153)

dim(accepted_2007_to_2018Q4)
colnames(accepted_2007_to_2018Q4)
head(accepted_2007_to_2018Q4)
t(accepted_2007_to_2018Q4[1, ])

#Checking how many missing values are present
accepted_2007_to_2018Q4 %>%
  summarise_all(~ sum(is.na(.))) %>%
  gather(key = "variable", value = "missing_count") %>%
  arrange(desc(missing_count)) %>%
  print(n = 151)

accepted_2007_to_2018Q4 %>% count(loan_status)

#Removing columns with more than 95% missing values
threshold <- 0.95 * nrow(accepted_2007_to_2018Q4)
missing_counts <- colSums(is.na(accepted_2007_to_2018Q4))
cols_to_drop <- names(missing_counts[missing_counts > threshold])
accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4 %>% select(-all_of(cols_to_drop))
cols_to_drop

dim(accepted_2007_to_2018Q4_clean)
t(accepted_2007_to_2018Q4_clean[1,])

#Removing loan statuses and changing to numerical values
accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>%
  filter(!loan_status %in% c("Current", "In Grace Period", "Late (16-30 days)", "Late (31-120
days)", NA)) %>%
  mutate(default = case_when(
    loan_status %in% c("Charged Off", "Default", "Does not meet the credit policy. Status:Charged
Off") ~ 1,
    loan_status %in% c("Fully Paid", "Does not meet the credit policy. Status:Fully Paid") ~ 0
  ))

accepted_2007_to_2018Q4_clean %>%
  select(loan_amnt, int_rate, annual_inc, dti) %>%
  summary()

# % of defaulters is 20%
accepted_2007_to_2018Q4_clean %>% count(default) %>% mutate(percent = n / sum(n) * 100)

#Create new feature for average fico score
accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>% mutate(fico_score_avg =
(fico_range_low + fico_range_high) / 2)

##### Graphs #####

#Default vs non default
accepted_2007_to_2018Q4_clean %>%
```

```

ggplot(aes(x = factor(default), fill = factor(default))) +
  geom_bar() +
  scale_fill_manual(values = c("0" = "green", "1" = "red"),
                    labels = c("Non-Default", "Default"),
                    name = "Status") +
  labs(title = "Default Distribution", x = "Default (0 = Non-Default, 1 = Default)", y = "Count")

#Histogram of loan amount spread
accepted_2007_to_2018Q4_clean %>%
  ggplot(aes(x = loan_amnt)) +
  geom_histogram(bins = 50, fill = "blue") +
  labs(title = "Loan Amount Distribution", x = "Loan Amount", y = "Count")

#boxplot of default vs non default by loan amount
accepted_2007_to_2018Q4_clean %>%
  ggplot(aes(x = factor(default), y = loan_amnt, fill = factor(default))) +
  geom_boxplot() +
  scale_fill_manual(values = c("0" = "green", "1" = "red"),
                    labels = c("Non-Default", "Default"),
                    name = "Status") +
  labs(title = "Loan Amount by Default Status", x = "Default", y = "Loan Amount")

#boxplot of interest rate vs default
accepted_2007_to_2018Q4_clean %>%
  ggplot(aes(x = factor(default), y = int_rate, fill = factor(default))) +
  geom_boxplot() +
  scale_fill_manual(values = c("0" = "green", "1" = "red")) +
  labs(title = "Interest Rate by Default Status", x = "Non Default (0) Default(1)", y = "Interest
Rate (%)")

# purpose of Loan vs Default Rate
accepted_2007_to_2018Q4_clean %>%
  filter(!is.na(purpose)) %>%
  ggplot(aes(x = fct_infreq(purpose), fill = factor(default))) +
  geom_bar(position = "fill") +
  scale_fill_manual(values = c("0" = "green", "1" = "red"),
                    labels = c("Non-Default", "Default"),
                    name = "Status") +
  coord_flip() +
  labs(title = "Proportion of Default by Loan Purpose",
       x = "Loan Purpose",
       y = "Proportion")

#barplot Home ownership vs default rate
accepted_2007_to_2018Q4_clean %>%
  filter(!is.na(home_ownership)) %>%
  ggplot(aes(x = fct_infreq(home_ownership), fill = factor(default))) +
  geom_bar(position = "fill") +
  scale_fill_manual(values = c("0" = "green", "1" = "red"),
                    labels = c("Non-Default", "Default"),
                    name = "Status") +
  labs(title = "Proportion of Default by Home Ownership",
       x = "Home Ownership",
       y = "Proportion")

#Comparing DTI to default rates plot
accepted_2007_to_2018Q4_clean %>%
  mutate(dti_bin = cut(dti,
                       breaks = c(0, 10, 20, 30, 40, 50, Inf),
                       labels = c("0-10", "10-20", "20-30", "30-40", "40-50", "50+"),
                       right = FALSE)) %>%
  ggplot(aes(x = dti_bin, fill = factor(default))) +
  geom_bar(position = "fill") +
  scale_fill_manual(values = c("0" = "green", "1" = "red"),
                    labels = c("Non-Default", "Default"),
                    name = "Status") +
  labs(title = "Proportion of Defaults Across DTI Bins",
       x = "DTI Range",
       y = "Proportion")

```

```

#FICO score compared to default status
accepted_2007_to_2018Q4_clean %>%
  filter(!is.na(fico_range_high)) %>%
  ggplot(aes(x = factor(default), y = fico_range_high, fill = factor(default))) +
  geom_boxplot() +
  scale_fill_manual(values = c("0" = "green", "1" = "red")) +
  labs(title = "FICO Score (High) by Default Status",
       x = "Default",
       y = "FICO Range High")

#revolving utilization vs default status (not so useful)

accepted_2007_to_2018Q4_clean %>% filter(!is.na(revol_util)) %>% mutate(revol_util_capped =
ifelse(revol_util > 150, 150, revol_util)) %>%
  ggplot(aes(x = factor(default), y = revol_util_capped, fill = factor(default))) +
  geom_boxplot() +
  scale_fill_manual(values = c("0" = "green", "1" = "red")) +
  labs(title = "Revolving Utilization (Capped at 150%) by Default Status",
       x = "Default",
       y = "Revolving Utilization (%)")

#Employment length vs default rate
accepted_2007_to_2018Q4_clean %>%
  group_by(emp_length) %>%
  summarise(default_rate = mean(default)) %>%
  ggplot(aes(x = reorder(emp_length, default_rate), y = default_rate)) +
  geom_col(fill = "blue") +
  coord_flip() +
  labs(title = "Default Rate by Employment Length", x = "Employment Length", y = "Default Rate")

#Loan term length vs default rate
accepted_2007_to_2018Q4_clean %>%
  group_by(term) %>%
  summarise(default_rate = mean(default)) %>%
  ggplot(aes(x = term, y = default_rate, fill = term)) +
  geom_col() +
  labs(title = "Default Rate by Loan Term", x = "Loan Term", y = "Default Rate")

##### Dropping columns #####

columns_to_keep <- c("loan_amnt", "term", "int_rate", "installment",
                    "annual_inc", "dti", "emp_length", "home_ownership",
                    "purpose", "verification_status",
                    "fico_score_avg", "revol_util", "default")

accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>%
  select(all_of(columns_to_keep))

dim(accepted_2007_to_2018Q4_clean)
head(accepted_2007_to_2018Q4_clean)
t(accepted_2007_to_2018Q4_clean[1, ])

#Checking for empty values
colSums(is.na(accepted_2007_to_2018Q4_clean))

#Impute numeric columns with median
accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>% mutate(
  annual_inc = ifelse(is.na(annual_inc), median(annual_inc, na.rm = TRUE), annual_inc),
  dti = ifelse(is.na(dti), median(dti, na.rm = TRUE), dti),
  revol_util = ifelse(is.na(revol_util), median(revol_util, na.rm = TRUE), revol_util)
)

#Impute categorical column emp_length with "Unknown"
accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>% mutate(emp_length =
ifelse(is.na(emp_length), "Unknown", emp_length))

sum(duplicated(accepted_2007_to_2018Q4_clean))

```

```

summary(accepted_2007_to_2018Q4_clean)

#Capping annual_inc to $500,000
accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>%
  mutate(annual_inc = ifelse(annual_inc > 500000, 500000, annual_inc))

#Checking how many dti's were impossible values
sum(accepted_2007_to_2018Q4_clean$dti < 0, na.rm = TRUE)
sum(accepted_2007_to_2018Q4_clean$dti > 100, na.rm = TRUE)

#changing dti outliers to median
median_dti <- median(accepted_2007_to_2018Q4_clean$dti, na.rm = TRUE)

accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>%
  mutate(dti = ifelse(dti < 0 | dti > 100, median_dti, dti))

#checking revol_util values
sum(accepted_2007_to_2018Q4_clean$revol_util > 150, na.rm = TRUE)

#capping revol_util at 150%
accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>%
  mutate(revol_util = ifelse(revol_util > 150, 150, revol_util))

#correlation matrix
numeric_features <- accepted_2007_to_2018Q4_clean %>%
  select(where(is.numeric))
cor_matrix <- cor(numeric_features, use = "complete.obs")
print(cor_matrix)

#correlation map
corrplot(cor_matrix,
  method = "color",
  type = "upper",
  tl.col = "black",
  tl.cex = 0.8,
  addCoef.col = "black",
  number.cex = 0.7)

colnames(accepted_2007_to_2018Q4_clean)
unique(accepted_2007_to_2018Q4_clean$purpose)
unique(accepted_2007_to_2018Q4_clean$default)

summary(accepted_2007_to_2018Q4_clean)

colnames(accepted_2007_to_2018Q4_clean)
dim(accepted_2007_to_2018Q4_clean)

write_csv(accepted_2007_to_2018Q4_clean, "cleaned_lendingclub.csv")

library(tidyverse)
library(ggplot2)
library(corrplot)

accepted_2007_to_2018Q4 <- read_csv("accepted_2007_to_2018q4.csv/accepted_2007_to_2018Q4.csv")
accepted_2007_to_2018Q4

lc_data_def <- read_csv("Lending Club Data Dictionary Approved.csv")
lc_data_def

lc_data_def %>%select(`LoanStatNew`, Description) %>% print(n = 153)

dim(accepted_2007_to_2018Q4)
colnames(accepted_2007_to_2018Q4)
head(accepted_2007_to_2018Q4)
t(accepted_2007_to_2018Q4[1, ])

```

```

#Checking how many missing values are present
accepted_2007_to_2018Q4 %>%
  summarise_all(~ sum(is.na(.))) %>%
  gather(key = "variable", value = "missing_count") %>%
  arrange(desc(missing_count)) %>%
  print(n = 151)

accepted_2007_to_2018Q4 %>% count(loan_status)

#Removing columns with more than 95% missing values
threshold <- 0.95 * nrow(accepted_2007_to_2018Q4)
missing_counts <- colSums(is.na(accepted_2007_to_2018Q4))
cols_to_drop <- names(missing_counts[missing_counts > threshold])
accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4 %>% select(-all_of(cols_to_drop))
cols_to_drop

dim(accepted_2007_to_2018Q4_clean)
t(accepted_2007_to_2018Q4_clean[1,])

#Removing loan statuses and changing to numerical values
accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>%
  filter(!loan_status %in% c("Current", "In Grace Period", "Late (16-30 days)", "Late (31-120
days)", NA)) %>%
  mutate(default = case_when(
    loan_status %in% c("Charged Off", "Default", "Does not meet the credit policy. Status:Charged
Off") ~ 1,
    loan_status %in% c("Fully Paid", "Does not meet the credit policy. Status:Fully Paid") ~ 0
  ))

accepted_2007_to_2018Q4_clean %>%
  select(loan_amnt, int_rate, annual_inc, dti) %>%
  summary()

accepted_2007_to_2018Q4_clean %>% count(default) %>% mutate(percent = n / sum(n) * 100)

#Create new feature for average fico score
accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>% mutate(fico_score_avg =
(fico_range_low + fico_range_high) / 2)

##### Graphs #####

#Default vs non default
accepted_2007_to_2018Q4_clean %>%
  ggplot(aes(x = factor(default), fill = factor(default))) +
  geom_bar() +
  scale_fill_manual(values = c("0" = "green", "1" = "red"),
    labels = c("Non-Default", "Default"),
    name = "Status") +
  labs(title = "Default Distribution", x = "Default (0 = Non-Default, 1 = Default)", y = "Count")

#Histogram of loan amount spread
accepted_2007_to_2018Q4_clean %>%
  ggplot(aes(x = loan_amnt)) +
  geom_histogram(bins = 50, fill = "blue") +
  labs(title = "Loan Amount Distribution", x = "Loan Amount", y = "Count")

#boxplot of default vs non default by loan amount
accepted_2007_to_2018Q4_clean %>%
  ggplot(aes(x = factor(default), y = loan_amnt, fill = factor(default))) +
  geom_boxplot() +
  scale_fill_manual(values = c("0" = "green", "1" = "red"),
    labels = c("Non-Default", "Default"),
    name = "Status") +
  labs(title = "Loan Amount by Default Status", x = "Default", y = "Loan Amount")

#boxplot of interest rate vs default

```



```

accepted_2007_to_2018Q4_clean %>%
  ggplot(aes(x = factor(default), y = int_rate, fill = factor(default))) +
  geom_boxplot() +
  scale_fill_manual(values = c("0" = "green", "1" = "red")) +
  labs(title = "Interest Rate by Default Status", x = "Non Default (0) Default(1)", y = "Interest
Rate (%)")

#purpose of Loan vs Default Rate
accepted_2007_to_2018Q4_clean %>%
  filter(!is.na(purpose)) %>%
  ggplot(aes(x = fct_infreq(purpose), fill = factor(default))) +
  geom_bar(position = "fill") +
  scale_fill_manual(values = c("0" = "green", "1" = "red"),
                    labels = c("Non-Default", "Default"),
                    name = "Status") +
  coord_flip() +
  labs(title = "Proportion of Default by Loan Purpose",
       x = "Loan Purpose",
       y = "Proportion")

#barplot Home ownership vs default rate
accepted_2007_to_2018Q4_clean %>%
  filter(!is.na(home_ownership)) %>%
  ggplot(aes(x = fct_infreq(home_ownership), fill = factor(default))) +
  geom_bar(position = "fill") +
  scale_fill_manual(values = c("0" = "green", "1" = "red"),
                    labels = c("Non-Default", "Default"),
                    name = "Status") +
  labs(title = "Proportion of Default by Home Ownership",
       x = "Home Ownership",
       y = "Proportion")

#Comparing DTI to default rates plot
accepted_2007_to_2018Q4_clean %>%
  mutate(dti_bin = cut(dti,
                       breaks = c(0, 10, 20, 30, 40, 50, Inf),
                       labels = c("0-10", "10-20", "20-30", "30-40", "40-50", "50+"),
                       right = FALSE)) %>%
  ggplot(aes(x = dti_bin, fill = factor(default))) +
  geom_bar(position = "fill") +
  scale_fill_manual(values = c("0" = "green", "1" = "red"),
                    labels = c("Non-Default", "Default"),
                    name = "Status") +
  labs(title = "Proportion of Defaults Across DTI Bins",
       x = "DTI Range",
       y = "Proportion")

#FICO score compared to default status
accepted_2007_to_2018Q4_clean %>%
  filter(!is.na(fico_range_high)) %>%
  ggplot(aes(x = factor(default), y = fico_range_high, fill = factor(default))) +
  geom_boxplot() +
  scale_fill_manual(values = c("0" = "green", "1" = "red")) +
  labs(title = "FICO Score (High) by Default Status",
       x = "Default",
       y = "FICO Range High")

#revolving utilization vs default status

accepted_2007_to_2018Q4_clean %>% filter(!is.na(revol_util)) %>% mutate(revol_util_capped =
ifelse(revol_util > 150, 150, revol_util)) %>%
  ggplot(aes(x = factor(default), y = revol_util_capped, fill = factor(default))) +
  geom_boxplot() +
  scale_fill_manual(values = c("0" = "green", "1" = "red")) +
  labs(title = "Revolving Utilization (Capped at 150%) by Default Status",
       x = "Default",
       y = "Revolving Utilization (%)")

# Employemnet length vs default rate

```

```

accepted_2007_to_2018Q4_clean %>%
  group_by(emp_length) %>%
  summarise(default_rate = mean(default)) %>%
  ggplot(aes(x = reorder(emp_length, default_rate), y = default_rate)) +
  geom_col(fill = "blue") +
  coord_flip() +
  labs(title = "Default Rate by Employment Length", x = "Employment Length", y = "Default Rate")

#Loan term length vs default rate
accepted_2007_to_2018Q4_clean %>%
  group_by(term) %>%
  summarise(default_rate = mean(default)) %>%
  ggplot(aes(x = term, y = default_rate, fill = term)) +
  geom_col() +
  labs(title = "Default Rate by Loan Term", x = "Loan Term", y = "Default Rate")

##### Dropping columns #####

columns_to_keep <- c("loan_amnt", "term", "int_rate", "installment",
                    "annual_inc", "dti", "emp_length", "home_ownership",
                    "purpose", "verification_status",
                    "fico_score_avg", "revol_util", "default")

accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>%
  select(all_of(columns_to_keep))

dim(accepted_2007_to_2018Q4_clean)
head(accepted_2007_to_2018Q4_clean)
t(accepted_2007_to_2018Q4_clean[1, ])

#Checking for empty values
colSums(is.na(accepted_2007_to_2018Q4_clean))

#Impute numeric columns with median
accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>% mutate(
  annual_inc = ifelse(is.na(annual_inc), median(annual_inc, na.rm = TRUE), annual_inc),
  dti = ifelse(is.na(dti), median(dti, na.rm = TRUE), dti),
  revol_util = ifelse(is.na(revol_util), median(revol_util, na.rm = TRUE), revol_util)
)

#Impute categorical column emp_length with "Unknown"
accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>% mutate(emp_length =
  ifelse(is.na(emp_length), "Unknown", emp_length))

sum(duplicated(accepted_2007_to_2018Q4_clean))

summary(accepted_2007_to_2018Q4_clean)

#Capping annual_inc to $500,000
accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>%
  mutate(annual_inc = ifelse(annual_inc > 500000, 500000, annual_inc))

#Checking how many dti's were impossible values
sum(accepted_2007_to_2018Q4_clean$dti < 0, na.rm = TRUE)
sum(accepted_2007_to_2018Q4_clean$dti > 100, na.rm = TRUE)

#changing dti outliers to median
median_dti <- median(accepted_2007_to_2018Q4_clean$dti, na.rm = TRUE)

accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>%
  mutate(dti = ifelse(dti < 0 | dti > 100, median_dti, dti))

#checking revol_util values
sum(accepted_2007_to_2018Q4_clean$revol_util > 150, na.rm = TRUE)

#capping revol_util at 150%
accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>%
  mutate(revol_util = ifelse(revol_util > 150, 150, revol_util))

```

```

#correlation matrix
numeric_features <- accepted_2007_to_2018Q4_clean %>%
  select(where(is.numeric))
cor_matrix <- cor(numeric_features, use = "complete.obs")
print(cor_matrix)

#correlation map
corrplot(cor_matrix,
          method = "color",
          type = "upper",
          tl.col = "black",
          tl.cex = 0.8,
          addCoef.col = "black",
          number.cex = 0.7)

colnames(accepted_2007_to_2018Q4_clean)
unique(accepted_2007_to_2018Q4_clean$purpose)
unique(accepted_2007_to_2018Q4_clean$default)

summary(accepted_2007_to_2018Q4_clean)

colnames(accepted_2007_to_2018Q4_clean)
dim(accepted_2007_to_2018Q4_clean)

write_csv(accepted_2007_to_2018Q4_clean, "cleaned_lendingclub.csv")

```

Appendix B – Jupyter notebook code for initial model training.

```

#!/usr/bin/env python
# coding: utf-8

# In[1]:

import pandas as pd

# Load the dataset
df = pd.read_csv("cleaned_lendingclub.csv")

# Check the shape and first rows
print(df.shape)
df.head()

# In[2]:

df.info()
print(df.shape)
df.isnull().sum()

# In[3]:

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
roc_auc_score, classification_report

# In[4]:

```

```

print(df['term'].unique())
print(df.columns.tolist())

# In[5]:

#Encoding variables

df['term'] = df['term'].astype(str).str.extract(r'(\d+)').astype(int)
print(df['term'].unique())

# In[6]:

emp_map = {
    "10+ years": 10, "9 years": 9, "8 years": 8, "7 years": 7,
    "6 years": 6, "5 years": 5, "4 years": 4, "3 years": 3,
    "2 years": 2, "1 year": 1, "< 1 year": 0, "Unknown": -1
}

df['emp_length'] = df['emp_length'].map(emp_map)
print(df['emp_length'].unique())

# In[7]:

df = pd.get_dummies(
    df,
    columns=['home_ownership', 'purpose', 'verification_status'],
    drop_first=True
)

print(df.shape)
print(df.columns.tolist()[:15]) # just to see first 15 columns

# In[8]:

from sklearn.model_selection import train_test_split

# In[9]:

X = df.drop('default', axis=1)
y = df['default']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

print(X_train.shape, X_test.shape)

# In[10]:

import numpy as np

corr_matrix = X.corr().abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
high_corr = [column for column in upper.columns if any(upper[column] > 0.9)]

print("Highly correlated features to consider dropping:", high_corr)

```

```

# In[11]:

X = X.drop('installment', axis=1)

# In[12]:

# Logistic Regression #

# In[13]:

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

# In[14]:

log_reg = LogisticRegression(max_iter=1000, class_weight='balanced')
log_reg.fit(X_train, y_train)

y_pred_log = log_reg.predict(X_test)

print(confusion_matrix(y_test, y_pred_log))
print(classification_report(y_test, y_pred_log))

# In[15]:

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# In[16]:

log_reg = LogisticRegression(max_iter=2000, class_weight='balanced')
log_reg.fit(X_train_scaled, y_train)

y_pred_log = log_reg.predict(X_test_scaled)

from sklearn.metrics import classification_report, confusion_matrix
print(confusion_matrix(y_test, y_pred_log))
print(classification_report(y_test, y_pred_log))

# In[17]:

##### Random Forest #####

# In[18]:

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

```

```

# In[19]:

rf = RandomForestClassifier(n_estimators=200, max_depth=None, random_state=42,
class_weight='balanced')
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)

print(confusion_matrix(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))

# In[20]:

from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

print("Before balancing:", y_train.value_counts())
print("After balancing:", y_train_res.value_counts())

# In[21]:

rf_balanced = RandomForestClassifier(
    n_estimators=100,
    max_depth=20,
    random_state=42,
    class_weight=None,
    n_jobs=-1
)

rf_balanced.fit(X_train_res, y_train_res)

y_pred_rf_balanced = rf_balanced.predict(X_test)

from sklearn.metrics import classification_report, confusion_matrix
print(confusion_matrix(y_test, y_pred_rf_balanced))
print(classification_report(y_test, y_pred_rf_balanced))

# In[22]:

# Testing new parameters for the model#

# In[23]:

from sklearn.model_selection import RandomizedSearchCV
import numpy as np

param_dist = {
    'n_estimators': [100, 200, 300, 500],
    'max_depth': [10, 20, 30, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2']
}

rf = RandomForestClassifier(random_state=42, n_jobs=-1)

random_search = RandomizedSearchCV(
    estimator=rf,

```

```

        param_distributions=param_dist,
        n_iter=20,
        cv=3,
        scoring='f1',
        verbose=1,
        random_state=42
    )

random_search.fit(X_train_res, y_train_res)

print("Best Parameters:", random_search.best_params_)

# In[24]:

# New rf tuned #

# In[25]:

rf_tuned = RandomForestClassifier(
    n_estimators=100,
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    max_features='sqrt',
    random_state=42,
    n_jobs=-1
)

rf_tuned.fit(X_train_res, y_train_res)

y_pred_rf_tuned = rf_tuned.predict(X_test)

from sklearn.metrics import confusion_matrix, classification_report
print(confusion_matrix(y_test, y_pred_rf_tuned))
print(classification_report(y_test, y_pred_rf_tuned))

# In[26]:

from sklearn.metrics import classification_report, confusion_matrix

y_prob_rf = rf_tuned.predict_proba(X_test)[:, 1]

threshold = 0.25
y_pred_thresh = (y_prob_rf >= threshold).astype(int)

print(confusion_matrix(y_test, y_pred_thresh))
print(classification_report(y_test, y_pred_thresh))

# In[28]:

import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import precision_recall_curve, f1_score

y_prob_rf = rf_tuned.predict_proba(X_test)[:, 1]

precision, recall, thresholds = precision_recall_curve(y_test, y_prob_rf)

f1_scores = 2 * (precision * recall) / (precision + recall)

```

```

best_index = np.argmax(f1_scores)
best_threshold = thresholds[best_index]

print("Best Threshold:", best_threshold)
print("Best F1 Score:", f1_scores[best_index])

plt.figure(figsize=(8, 6))
plt.plot(thresholds, precision[:-1], label="Precision")
plt.plot(thresholds, recall[:-1], label="Recall")
plt.plot(thresholds, f1_scores[:-1], label="F1 Score")
plt.axvline(best_threshold, color='red', linestyle='--', label=f"Best Threshold={best_threshold:.2f}")
plt.xlabel("Threshold")
plt.ylabel("Score")
plt.legend()
plt.show()

# In[29]:

import pandas as pd
import matplotlib.pyplot as plt

importances = rf_tuned.feature_importances_
feature_names = X_train.columns

feat_importances = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
feat_importances = feat_importances.sort_values(by='Importance', ascending=False)

print(feat_importances.head(20))

plt.figure(figsize=(10, 6))
plt.barh(feat_importances['Feature'].head(20)[::-1], feat_importances['Importance'].head(20)[::-1])
plt.xlabel("Feature Importance")
plt.ylabel("Feature")
plt.title("Top 20 Feature Importances (Random Forest)")
plt.show()

# In[ ]:

# In[30]:

from imblearn.combine import SMOTEENN

smote_enn = SMOTEENN(random_state=42)
X_train_res2, y_train_res2 = smote_enn.fit_resample(X_train, y_train)

print("Before:", y_train.value_counts())
print("After:", y_train_res2.value_counts())

# In[31]:

rf_smoteenn = RandomForestClassifier(
    n_estimators=100,
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    max_features='sqrt',
    random_state=42,

```



```

        n_jobs=-1
    )

    rf_smoteenn.fit(X_train_res2, y_train_res2)

    y_prob_smoteenn = rf_smoteenn.predict_proba(X_test)[:, 1]
    y_pred_smoteenn = (y_prob_smoteenn >= 0.25).astype(int)

    from sklearn.metrics import confusion_matrix, classification_report
    print(confusion_matrix(y_test, y_pred_smoteenn))
    print(classification_report(y_test, y_pred_smoteenn))

# In[32]:

thresholds = [0.25, 0.30, 0.35, 0.40]
results = []

for t in thresholds:
    y_pred = (y_prob_smoteenn >= t).astype(int)
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    results.append([t, acc, prec, rec, f1])

df_results = pd.DataFrame(results, columns=["Threshold", "Accuracy", "Precision", "Recall", "F1-
score"])
print(df_results)

# In[33]:

# XGBoost model #

# In[34]:

get_ipython().system('pip install xgboost')

# In[35]:

from xgboost import XGBClassifier
from imblearn.combine import SMOTEENN
from sklearn.model_selection import RandomizedSearchCV
from sklearn.metrics import classification_report, confusion_matrix, precision_recall_curve
import numpy as np
import matplotlib.pyplot as plt

# In[36]:

xgb = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)

param_dist = {
    'n_estimators': [100, 200, 300],
    'max_depth': [3, 5, 7, 10],
    'learning_rate': [0.01, 0.05, 0.1, 0.2],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0]
}

xgb_search = RandomizedSearchCV(

```

```

        estimator=xgb,
        param_distributions=param_dist,
        n_iter=20,
        scoring='f1',
        cv=3,
        verbose=1,
        random_state=42,
        n_jobs=-1
    )

xgb_search.fit(X_train_res2, y_train_res2)

print("Best Parameters:", xgb_search.best_params_)

# In[37]:

from xgboost import XGBClassifier

xgb_best = XGBClassifier(
    n_estimators=200,
    max_depth=10,
    learning_rate=0.01,
    subsample=0.6,
    colsample_bytree=0.6,
    eval_metric='logloss',
    use_label_encoder=False,
    random_state=42
)

xgb_best.fit(X_train_res2, y_train_res2)

# In[38]:

xgb_best = xgb_search.best_estimator_
print(xgb_best.get_params())

# In[39]:

y_probs = xgb_best.predict_proba(X_test)[: , 1]

# In[40]:

from sklearn.metrics import precision_recall_curve
import numpy as np

precision, recall, thresholds = precision_recall_curve(y_test, y_probs)
f1_scores = np.where((precision + recall) == 0, 0, 2 * (precision * recall) / (precision + recall))

best_index = np.argmax(f1_scores)
best_threshold = thresholds[best_index]

print(f"Best Threshold: {best_threshold:.2f}")
print(f"Best F1 Score: {f1_scores[best_index]:.2f}")

# In[41]:

y_pred_thresh = (y_probs >= best_threshold).astype(int)

```

```

# In[42]:

import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import precision_recall_curve

precision, recall, thresholds = precision_recall_curve(y_test, y_probs)
f1_scores = 2 * (precision * recall) / (precision + recall)
f1_scores = np.where((precision + recall) == 0, 0, f1_scores)

best_index = np.argmax(f1_scores)
best_threshold = thresholds[best_index]

print("Best Threshold:", best_threshold)
print("Best F1 Score:", f1_scores[best_index])

plt.figure(figsize=(8, 6))
plt.plot(thresholds, precision[:-1], label="Precision")
plt.plot(thresholds, recall[:-1], label="Recall")
plt.plot(thresholds, f1_scores[:-1], label="F1 Score")
plt.axvline(best_threshold, color='green', linestyle='--', label=f"Best Threshold={best_threshold:.2f}")
plt.xlabel("Threshold")
plt.ylabel("Score")
plt.title("XGBoost: Precision, Recall, and F1 vs Threshold")
plt.legend()
plt.grid(True)
plt.show()

# In[43]:

from sklearn.metrics import confusion_matrix, classification_report

print(confusion_matrix(y_test, y_pred_thresh))
print(classification_report(y_test, y_pred_thresh))

# In[44]:

y_pred_thresh_025 = (y_probs >= 0.25).astype(int)
print(confusion_matrix(y_test, y_pred_thresh_025))
print(classification_report(y_test, y_pred_thresh_025))

# In[45]:

get_ipython().system('pip install shap')

# In[46]:

X_test = X_test.astype(float)

# In[47]:

X_train_res2 = pd.DataFrame(X_train_res2).astype(np.float64)
X_test = pd.DataFrame(X_test).astype(np.float64)

# In[48]:

```

```

import shap

explainer = shap.Explainer(xgb_best, X_train_res2)
shap_values = explainer(X_train_res2)

# In[156]:

import joblib
joblib.dump(shap_values, "shap_values.pkl")

# In[158]:

import os

os.path.getsize("shap_values.pkl") / (1024*1024) # size in MB

# In[ ]:

import joblib
shap_values = joblib.load("shap_values.pkl")

# In[49]:

shap.summary_plot(shap_values, X_train_res2)

# In[50]:

#Testing PCA on Logistic Regression#

# In[51]:

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_res2)
X_test_scaled = scaler.transform(X_test)

# In[52]:

from sklearn.decomposition import PCA

pca = PCA(n_components=0.95, random_state=42)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

print("Original shape:", X_train_res2.shape)
print("After PCA:", X_train_pca.shape)

# In[53]:

from sklearn.linear_model import LogisticRegression

```

```

from sklearn.metrics import classification_report, confusion_matrix

lr_pca = LogisticRegression(max_iter=1000, random_state=42)
lr_pca.fit(X_train_pca, y_train_res2)

y_prob = lr_pca.predict_proba(X_test_pca)[:, 1]

# In[54]:

from sklearn.metrics import precision_recall_curve
import numpy as np

precision, recall, thresholds = precision_recall_curve(y_test, y_prob)
f1_scores = 2 * (precision * recall) / (precision + recall + 1e-6)
best_idx = np.argmax(f1_scores)

best_threshold = thresholds[best_idx]
print("Best Threshold:", best_threshold)
print("Best F1 Score:", f1_scores[best_idx])

# In[55]:

y_pred = (y_prob >= best_threshold).astype(int)
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

# In[56]:

#Trying PCA with XGBoost#

# In[57]:

from xgboost import XGBClassifier

xgb_pca = XGBClassifier(
    n_estimators=200,
    max_depth=10,
    learning_rate=0.01,
    subsample=0.6,
    colsample_bytree=0.6,
    eval_metric='logloss',
    use_label_encoder=False,
    random_state=42
)

xgb_pca.fit(X_train_pca, y_train_res2)

# In[58]:

y_prob_xgb_pca = xgb_pca.predict_proba(X_test_pca)[:, 1]

# In[59]:

precision, recall, thresholds = precision_recall_curve(y_test, y_prob_xgb_pca)
f1_scores = 2 * (precision * recall) / (precision + recall + 1e-6)

best_idx = np.argmax(f1_scores)

```

```

best_threshold = thresholds[best_idx]

print("Best Threshold:", best_threshold)
print("Best F1 Score:", f1_scores[best_idx])

# In[60]:

y_pred_xgb_pca = (y_prob_xgb_pca >= best_threshold).astype(int)

from sklearn.metrics import classification_report, confusion_matrix

print(confusion_matrix(y_test, y_pred_xgb_pca))
print(classification_report(y_test, y_pred_xgb_pca))

# In[61]:

# Adding lightGBM#

# In[62]:

pip install lightgbm

# In[63]:

import lightgbm as lgb

lgbm = lgb.LGBMClassifier(
    n_estimators=200,
    max_depth=10,
    learning_rate=0.01,
    subsample=0.6,
    colsample_bytree=0.6,
    random_state=42
)

lgbm.fit(X_train_res2, y_train_res2)

# In[64]:

y_prob_lgbm = lgbm.predict_proba(X_test)[:, 1]

# In[65]:

from sklearn.metrics import precision_recall_curve
import numpy as np

precision, recall, thresholds = precision_recall_curve(y_test, y_prob_lgbm)
f1_scores = 2 * (precision * recall) / (precision + recall + 1e-6)

best_idx = np.argmax(f1_scores)
best_threshold = thresholds[best_idx]

print("Best Threshold:", best_threshold)
print("Best F1 Score:", f1_scores[best_idx])

# In[66]:

```

```

from sklearn.metrics import classification_report, confusion_matrix

y_pred_lgbm = (y_prob_lgbm >= best_threshold).astype(int)

print(confusion_matrix(y_test, y_pred_lgbm))
print(classification_report(y_test, y_pred_lgbm))

# In[67]:

shap_df = pd.DataFrame(shap_values.values, columns=X_train_res2.columns)
mean_shap = shap_df.abs().mean().sort_values(ascending=False)

top_n = 20
top_features = mean_shap.head(top_n).index.tolist()
print(top_features)

# In[68]:

X_train_top = X_train_res2[top_features]
X_test_top = X_test[top_features]

# In[69]:

import lightgbm as lgb

lgbm_shap = lgb.LGBMClassifier(
    n_estimators=200,
    max_depth=10,
    learning_rate=0.01,
    subsample=0.6,
    colsample_bytree=0.6,
    random_state=42
)

lgbm_shap.fit(X_train_top, y_train_res2)

# In[70]:

from sklearn.metrics import precision_recall_curve
import numpy as np

y_prob_shap = lgbm_shap.predict_proba(X_test_top)[: , 1]

precision, recall, thresholds = precision_recall_curve(y_test, y_prob_shap)
f1_scores = 2 * (precision * recall) / (precision + recall + 1e-6)

best_idx = np.argmax(f1_scores)
best_threshold = thresholds[best_idx]

print("Best Threshold:", best_threshold)
print("Best F1 Score:", f1_scores[best_idx])

# In[71]:

from sklearn.metrics import classification_report, confusion_matrix

y_pred_shap = (y_prob_shap >= best_threshold).astype(int)

```

```

print(confusion_matrix(y_test, y_pred_shap))
print(classification_report(y_test, y_pred_shap))

# In[160]:

get_ipython().run_line_magic('who', '')

# In[73]:

# AUC-ROC scores#

# In[74]:

from sklearn.metrics import roc_auc_score

aucs = {}

y_proba_lr_pca = lr_pca.predict_proba(X_test_pca)[: , 1]
aucs["LR + PCA"] = roc_auc_score(y_test, y_proba_lr_pca)

aucs["XGBoost + PCA"] = roc_auc_score(y_test, y_prob_xgb_pca)

aucs["LightGBM"] = roc_auc_score(y_test, y_prob_lgbm)

aucs["XGBoost"] = roc_auc_score(y_test, y_prob)

for k, v in aucs.items():
    print(f"{k} AUC-ROC: {v:.4f}")

# In[75]:

from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))

#Logistic Regression + PCA
fpr, tpr, _ = roc_curve(y_test, y_proba_lr_pca)
plt.plot(fpr, tpr, label=f'LR + PCA (AUC = {roc_auc_score(y_test, y_proba_lr_pca):.3f})')

#XGBoost + PCA
fpr, tpr, _ = roc_curve(y_test, y_prob_xgb_pca)
plt.plot(fpr, tpr, label=f'XGBoost + PCA (AUC = {roc_auc_score(y_test, y_prob_xgb_pca):.3f})')

#LightGBM
fpr, tpr, _ = roc_curve(y_test, y_prob_lgbm)
plt.plot(fpr, tpr, label=f'LightGBM (AUC = {roc_auc_score(y_test, y_prob_lgbm):.3f})')

#XGBoost
fpr, tpr, _ = roc_curve(y_test, y_prob)
plt.plot(fpr, tpr, label=f'XGBoost (AUC = {roc_auc_score(y_test, y_prob):.3f})')

plt.plot([0,1], [0,1], 'k--', label='Random Guess (AUC = 0.500)') # Diagonal line for random
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate (Recall)')
plt.title('ROC Curves for Loan Default Prediction Models')
plt.legend()
plt.grid(True)
plt.show()

```



```
# In[76]:  
  
print(X_train.columns.tolist())  
print(X_test.columns.tolist())  
#!/usr/bin/env python  
# coding: utf-8
```

Appendix C – R Code for enhanced EDA

```
library(tidyverse)
library(ggplot2)
library(corrplot)
library(lubridate)
library(scales)

accepted_2007_to_2018Q4 <- read_csv("accepted_2007_to_2018q4.csv/accepted_2007_to_2018Q4.csv")

threshold <- 0.95 * nrow(accepted_2007_to_2018Q4)
missing_counts <- colSums(is.na(accepted_2007_to_2018Q4))
cols_to_drop <- names(missing_counts[missing_counts > threshold])
accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4 %>% select(-all_of(cols_to_drop))

accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>%
  filter(!loan_status %in% c("Current", "In Grace Period", "Late (16-30 days)", "Late (31-120
days)", NA)) %>%
  mutate(default = case_when(
    loan_status %in% c("Charged Off", "Default", "Does not meet the credit policy. Status:Charged
Off") ~ 1,
    loan_status %in% c("Fully Paid", "Does not meet the credit policy. Status:Fully Paid") ~ 0
  ))

accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>%
  mutate(fico_score_avg = (fico_range_low + fico_range_high) / 2)

columns_to_keep <- c("loan_amnt", "term", "int_rate", "installment",
  "annual_inc", "dti", "emp_length", "home_ownership",
  "purpose", "verification_status", "fico_score_avg",
  "revol_util", "default", "issue_d", "mths_since_last_delinq",
  "open_acc", "pub_rec", "total_acc")

accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>%
  select(all_of(columns_to_keep))

accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>%
  mutate(
    issue_year = year(parse_date_time(issue_d, "b-Y")),
    issue_month = month(parse_date_time(issue_d, "b-Y")),

    mths_since_last_delinq = ifelse(is.na(mths_since_last_delinq), 999, mths_since_last_delinq)
  ) %>%
  select(-issue_d)

accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>% mutate(
  annual_inc = ifelse(is.na(annual_inc), median(annual_inc, na.rm = TRUE), annual_inc),
  dti = ifelse(is.na(dti), median(dti, na.rm = TRUE), dti),
  revol_util = ifelse(is.na(revol_util), median(revol_util, na.rm = TRUE), revol_util),
  open_acc = ifelse(is.na(open_acc), median(open_acc, na.rm = TRUE), open_acc),
  pub_rec = ifelse(is.na(pub_rec), median(pub_rec, na.rm = TRUE), pub_rec),
  total_acc = ifelse(is.na(total_acc), median(total_acc, na.rm = TRUE), total_acc)
)

#Impute categorical column emp_length with "Unknown"
accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>%
  mutate(emp_length = ifelse(is.na(emp_length), "Unknown", emp_length))

#Outlier handling
accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>%
  mutate(annual_inc = ifelse(annual_inc > 500000, 500000, annual_inc))

median_dti <- median(accepted_2007_to_2018Q4_clean$dti, na.rm = TRUE)
accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>%
  mutate(dti = ifelse(dti < 0 | dti > 100, median_dti, dti))

accepted_2007_to_2018Q4_clean <- accepted_2007_to_2018Q4_clean %>%
```

```

mutate(revol_util = ifelse(revol_util > 150, 150, revol_util))

dat <- accepted_2007_to_2018Q4_clean

# aggregate to default rate per issue year
issue_year_rates <- dat %>%
  dplyr::filter(!is.na(issue_year), issue_year >= 2007, issue_year <= 2018) %>%
  dplyr::group_by(issue_year) %>%
  dplyr::summarise(
    n = n(),
    defaults = sum(default == 1, na.rm = TRUE),
    default_rate = defaults / n,
    .groups = "drop"
  )

#Plot:line + points, percent y-axis
p_issue_year <- ggplot(issue_year_rates, aes(x = issue_year, y = default_rate)) +
  geom_line() +
  geom_point(size = 2) +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  scale_x_continuous(breaks = 2007:2018) +
  labs(
    title = "Default Rate by Issue Year",
    x = "Issue Year",
    y = "Default Rate",
    caption = "Rate = (# defaults / # loans) within each issue year"
  ) +
  theme_minimal(base_size = 12)
#Plot of rate of defaults each year
print(p_issue_year)

write_csv(accepted_2007_to_2018Q4_clean, "cleaned_lendingclub_enhanced.csv")

dim(accepted_2007_to_2018Q4_clean)
colnames(accepted_2007_to_2018Q4_clean)

```

Appendix D – Jupyter notebook for final model evaluation

```

#!/usr/bin/env python
# coding: utf-8

# In[7]:

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, precision_recall_curve,
f1_score, roc_auc_score, roc_curve
import xgboost as xgb
import lightgbm as lgb
import matplotlib.pyplot as plt
import warnings

warnings.filterwarnings('ignore')

df = pd.read_csv("cleaned_lendingclub_enhanced.csv")

df['term'] = df['term'].astype(str).str.extract(r'(\d+)').astype(int)
emp_map = {
    "10+ years": 10, "9 years": 9, "8 years": 8, "7 years": 7,

```

```

        "6 years": 6, "5 years": 5, "4 years": 4, "3 years": 3,
        "2 years": 2, "1 year": 1, "< 1 year": 0, "Unknown": -1
    }
    df['emp_length'] = df['emp_length'].map(emp_map)

    df['loan_to_income'] = df['loan_amnt'] / (df['annual_inc'] + 1)
    df['installment_to_income'] = df['installment'] / (df['annual_inc'] + 1)
    df['term_to_emp_length'] = df['term'] / (df['emp_length'] + 1.001)
    df['fico_by_revol_util'] = df['fico_score_avg'] / (df['revol_util'] + 1)
    df['income_by_fico'] = df['annual_inc'] / (df['fico_score_avg'] + 1)

    df['dti_x_int_rate'] = df['dti'] * df['int_rate']
    df['fico_score_sq'] = df['fico_score_avg']**2

    df['fico_category'] = pd.cut(df['fico_score_avg'],
                                bins=[0, 639, 699, 749, 900],
                                labels=['Poor', 'Fair', 'Good', 'Excellent'])

    df['int_rate_category'] = pd.cut(df['int_rate'],
                                    bins=[0, 10, 15, 20, 100],
                                    labels=['Low', 'Medium', 'High', 'Very High'])

    df = pd.get_dummies(
        df,
        columns=['home_ownership', 'purpose', 'verification_status', 'fico_category',
        'int_rate_category'],
        drop_first=True
    )

    X = df.drop('default', axis=1)
    y = df['default']

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=42, stratify=y
    )

    model_results = {}
    final_predictions = {}

    print("--- Training and Tuning Logistic Regression ---")
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    lr = LogisticRegression(class_weight='balanced', random_state=42, max_iter=2000, n_jobs=-1)
    param_dist_lr = {'C': np.logspace(-3, 2, 10), 'solver': ['saga']}
    random_search_lr = RandomizedSearchCV(lr, param_dist_lr, n_iter=10, scoring='f1_macro', cv=3,
    verbose=1, random_state=42, n_jobs=-1)
    random_search_lr.fit(X_train_scaled, y_train)
    best_lr = random_search_lr.best_estimator_

    y_prob_lr = best_lr.predict_proba(X_test_scaled)[: , 1]
    precision, recall, thresholds = precision_recall_curve(y_test, y_prob_lr)
    f1_scores = 2 * (precision * recall) / (precision + recall + 1e-6)
    best_threshold_lr = thresholds[np.argmax(f1_scores)]
    y_pred_lr_tuned = (y_prob_lr >= best_threshold_lr).astype(int)

    print("\nBest Logistic Regression Parameters:", random_search_lr.best_params_)
    print("Best Threshold for LR:", best_threshold_lr)
    print("\nLogistic Regression Tuned Classification Report:\n", classification_report(y_test,
    y_pred_lr_tuned))
    model_results['Logistic Regression'] = {'F1-Score (Default=1)': f1_score(y_test, y_pred_lr_tuned,
    pos_label=1), 'AUC-ROC': roc_auc_score(y_test, y_prob_lr)}
    final_predictions['Logistic Regression'] = y_pred_lr

    print("\n\n--- Training and Tuning Random Forest ---")
    rf = RandomForestClassifier(class_weight='balanced', random_state=42, n_jobs=-1)
    param_dist_rf = {'n_estimators': [100, 200], 'max_depth': [10, 15, 20], 'min_samples_split': [2,
    5], 'min_samples_leaf': [2, 4]}

```

```

random_search_rf = RandomizedSearchCV(rf, param_dist_rf, n_iter=10, scoring='f1_macro', cv=3,
verbose=1, random_state=42, n_jobs=-1)
random_search_rf.fit(X_train, y_train)
best_rf = random_search_rf.best_estimator_

y_prob_rf = best_rf.predict_proba(X_test)[: , 1]
precision, recall, thresholds = precision_recall_curve(y_test, y_prob_rf)
f1_scores = 2 * (precision * recall) / (precision + recall + 1e-6)
best_threshold_rf = thresholds[np.argmax(f1_scores)]
y_pred_rf_tuned = (y_prob_rf >= best_threshold_rf).astype(int)

print("\nBest Random Forest Parameters:", random_search_rf.best_params_)
print("Best Threshold for RF:", best_threshold_rf)
print("\nRandom Forest Tuned Classification Report:\n", classification_report(y_test,
y_pred_rf_tuned))
model_results['Random Forest'] = {'F1-Score (Default=1)': f1_score(y_test, y_pred_rf_tuned,
pos_label=1), 'AUC-ROC': roc_auc_score(y_test, y_prob_rf)}
final_predictions['Random Forest'] = y_prob_rf

print("\n\n--- Training and Tuning XGBoost ---")
scale_pos_weight = y_train.value_counts()[0] / y_train.value_counts()[1]
xgb_clf = xgb.XGBClassifier(objective='binary:logistic', eval_metric='logloss',
use_label_encoder=False, random_state=42)
param_dist_xgb = {'n_estimators': [200, 300], 'max_depth': [7, 10, 12], 'learning_rate': [0.05,
0.1], 'subsample': [0.7, 0.8], 'colsample_bytree': [0.7, 0.8], 'scale_pos_weight':
[scale_pos_weight]}
random_search_xgb = RandomizedSearchCV(xgb_clf, param_dist_xgb, n_iter=15, scoring='f1_macro',
cv=3, verbose=1, random_state=42, n_jobs=-1)
random_search_xgb.fit(X_train, y_train)
best_xgb = random_search_xgb.best_estimator_

y_prob_xgb = best_xgb.predict_proba(X_test)[: , 1]
precision, recall, thresholds = precision_recall_curve(y_test, y_prob_xgb)
f1_scores = 2 * (precision * recall) / (precision + recall + 1e-6)
best_threshold_xgb = thresholds[np.argmax(f1_scores)]
y_pred_xgb_tuned = (y_prob_xgb >= best_threshold_xgb).astype(int)

print("\nBest XGBoost Parameters:", random_search_xgb.best_params_)
print("Best Threshold for XGBoost:", best_threshold_xgb)
print("\nXGBoost Tuned Classification Report:\n", classification_report(y_test, y_pred_xgb_tuned))
model_results['XGBoost'] = {'F1-Score (Default=1)': f1_score(y_test, y_pred_xgb_tuned,
pos_label=1), 'AUC-ROC': roc_auc_score(y_test, y_prob_xgb)}
final_predictions['XGBoost'] = y_prob_xgb

print("\n\n--- Training and Tuning LightGBM ---")
lgbm = lgb.LGBMClassifier(objective='binary', metric='logloss', random_state=42,
class_weight='balanced', n_jobs=-1)
param_dist_lgbm = {'n_estimators': [200, 300], 'learning_rate': [0.05, 0.1], 'num_leaves': [40,
50, 60], 'max_depth': [10, 12], 'subsample': [0.7, 0.8], 'colsample_bytree': [0.7, 0.8]}
random_search_lgbm = RandomizedSearchCV(lgbm, param_dist_lgbm, n_iter=15, scoring='f1_macro',
cv=3, verbose=1, random_state=42, n_jobs=-1)
random_search_lgbm.fit(X_train, y_train)
best_lgbm = random_search_lgbm.best_estimator_

y_prob_lgbm = best_lgbm.predict_proba(X_test)[: , 1]
precision, recall, thresholds = precision_recall_curve(y_test, y_prob_lgbm)
f1_scores = 2 * (precision * recall) / (precision + recall + 1e-6)
best_threshold_lgbm = thresholds[np.argmax(f1_scores)]
y_pred_lgbm_tuned = (y_prob_lgbm >= best_threshold_lgbm).astype(int)

print("\nBest LightGBM Parameters:", random_search_lgbm.best_params_)
print("Best Threshold for LightGBM:", best_threshold_lgbm)
print("\nLightGBM Tuned Classification Report:\n", classification_report(y_test,
y_pred_lgbm_tuned))
model_results['LightGBM'] = {'F1-Score (Default=1)': f1_score(y_test, y_pred_lgbm_tuned,
pos_label=1), 'AUC-ROC': roc_auc_score(y_test, y_prob_lgbm)}
final_predictions['LightGBM'] = y_prob_lgbm

print("\n\n--- Final Model Comparison ---")

```

```

summary_df = pd.DataFrame(model_results).T
summary_df = summary_df.sort_values(by='F1-Score (Default=1)', ascending=False)
print(summary_df)

plt.figure(figsize=(10, 8))
for model_name, y_prob in final_predictions.items():
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    auc = roc_auc_score(y_test, y_prob)
    plt.plot(fpr, tpr, label=f'{model_name} (AUC = {auc:.4f})')

plt.plot([0, 1], [0, 1], 'k--', label='Random Guess (AUC = 0.500)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate (Recall)')
plt.title('ROC Curves for All Tuned Models on Enhanced Feature Set')
plt.legend()
plt.grid(True)
plt.show()

# In[9]:

import shap

print("\n\n--- Generating SHAP Analysis for LightGBM Model ---")

explainer = shap.Explainer(best_lgbm)

X_test_sample = X_test.sample(n=50000, random_state=42)
shap_values = explainer(X_test_sample)

print("Displaying SHAP Summary Plot...")
shap.summary_plot(shap_values, X_test_sample, plot_type="bar", show=False)
plt.title("SHAP Feature Importance (Global)")
plt.show()

shap.summary_plot(shap_values, X_test_sample, show=False)
plt.title("SHAP Feature Impact Summary")
plt.show()

# In[11]:

from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, precision_recall_curve,
f1_score, roc_auc_score, roc_curve
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

lr_pipe = make_pipeline(StandardScaler(), best_lr)

estimators = [
    ('lr', lr_pipe),
    ('rf', best_rf),
    ('xgb', best_xgb),
    ('lgbm', best_lgbm)
]

meta_model = LogisticRegression(solver='liblinear', random_state=42)

stacking_clf = StackingClassifier(
    estimators=estimators,
    final_estimator=meta_model,
    cv=5,

```

```

        stack_method='predict_proba',
        passthrough=True,
        n_jobs=-1
    )

    stacking_clf.fit(X_train, y_train)

    y_prob_stack = stacking_clf.predict_proba(X_test)[: , 1]
    precision, recall, thresholds = precision_recall_curve(y_test, y_prob_stack)
    f1_scores = 2 * (precision * recall) / (precision + recall + 1e-6)
    best_threshold_stack = thresholds[np.argmax(f1_scores)]
    y_pred_stack_tuned = (y_prob_stack >= best_threshold_stack).astype(int)

    print("Best Threshold for Stacking Ensemble:", best_threshold_stack)
    print("Stacking Ensemble Tuned Classification Report:\n", classification_report(y_test,
    y_pred_stack_tuned))

    model_results['Stacking Ensemble'] = {'F1-Score (Default=1)': f1_score(y_test, y_pred_stack_tuned,
    pos_label=1), 'AUC-ROC': roc_auc_score(y_test, y_prob_stack)}
    final_predictions['Stacking Ensemble'] = y_prob_stack

    print("\n\n--- Final Model Comparison ---")
    summary_df = pd.DataFrame(model_results).T
    summary_df = summary_df.sort_values(by='F1-Score (Default=1)', ascending=False)
    print(summary_df)

    plt.figure(figsize=(10, 8))
    for model_name, y_prob in final_predictions.items():
        fpr, tpr, _ = roc_curve(y_test, y_prob)
        auc = roc_auc_score(y_test, y_prob)
        plt.plot(fpr, tpr, label=f'{model_name} (AUC = {auc:.4f})')

    plt.plot([0, 1], [0, 1], 'k--', label='Random Guess (AUC = 0.500)')
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate (Recall)')
    plt.title('ROC Curves for All Tuned Models (Including Stacking)')
    plt.legend()
    plt.grid(True)
    plt.show()

# In[19]:

get_ipython().system('jupyter nbconvert --to script Dissertation_Test_1.ipynb')

```